1 Algorithms

1.1 manhattan

```cpp
// gives farthest pair of points in manhattan distance in O(n*
2^d *d), d = dimension of the points
// bruteforce signs looking to maximize |x_p - x_q| + |y_p -
y_q|, example:
// max_{p, q \in P}[(x_p + (-x_q)) + (y_p + (-y_q))] = max_{p
\in P}(x_p + y_p) + max_{q \in P}(-x_q - y_p)
long long ans = 0;
for (int msk = 0; msk < (1 << d); msk++) {
    long long mx = LLONG_MIN, mn = LLONG_MAX;
    for (int i = 0; i < n; i++) {
        long long cur = 0;
        for (int j = 0; j < d; j++) {
            if (msk & (1 << j)) cur += p[i][j];
            else cur -= p[i][j];
        }
        mx = max(mx, cur);
        mn = min(mn, cur);
    }
    ans = max(ans, mx - mn);
}
struct point {
    long long x, y;
};
// Manhattan MST, cost of edges between points are its
manhattan distance
// Returns a list of edges in the format (weight, u, v).
// Passing this list to Kruskal algorithm will give the
Manhattan MST.
vector<tuple<long long, int, int>>
manhattan_mst_edges(vector<point> ps) {
    vector<int> ids(ps.size());
    iota(ids.begin(), ids.end(), 0);
    vector<tuple<long long, int, int>> edges;
    for (int rot = 0; rot < 4; rot++) { // for every rotation
        sort(ids.begin(), ids.end(), [&](int i, int j){
            return (ps[i].x + ps[i].y) < (ps[j].x + ps[j].y);
        });
        map<int, int, greater<int>> active; // (xs, id)
        for (auto i : ids) {
            for (auto it = active.lower_bound(ps[i].x); it !=
active.end();
            active.erase(it++)) {
                int j = it->second;
                if (ps[i].x - ps[i].y > ps[j].x - ps[j].y)
break;
                assert(ps[i].x >= ps[j].x && ps[i].y >=
ps[j].y);
                edges.push_back({(ps[i].x - ps[j].x) + (ps[i].y
- ps[j].y), i, j});
            }
            active[ps[i].x] = i;
        }
        for (auto &p : ps) { // rotate
            if (rot & 1) p.x *= -1;
            else swap(p.x, p.y);
        }
    }
    return edges;
}
```

1.2 ternary search

```cpp
template <typename T = double>
typename enable_if<is_floating_point<T>::value>
ternarySearch(function<T(T)> calculate, T left, T right, bool
searchMin = true,
              T epsilon = 1e-9) { // [left, right]
  while (right - left > epsilon) {
    T midA = left + (right - left) / 3, midB = right - (right -
left) / 3;
    T valueA = calculate(midA), valueB = calculate(midB);
    if (searchMin)
      valueA *= -1, valueB *= -1;
    if (valueA < valueB)
      left = midA;
    else
      right = midB;
  }
  return calculate(left);
}
template <typename T = int>
typename enable_if<is_integral<T>::value>
ternarySearch(function<T(T)> calculate, T left, T right, bool
searchMin = true,
              T epsilon = 5) { // [left, right]
  while (right - left > epsilon) {
    T midA = left + (right - left) / 3, midB = right - (right -
left) / 3;
    T valueA = calculate(midA), valueB = calculate(midB);
    if (searchMin)
      valueA *= -1, valueB *= -1;
    if (valueA < valueB)
      left = midA;
    else
      right = midB;
  }
  T answer = calculate(left);
  for (T i = left + 1; i <= right; i++) {
    T value = calculate(i);
    if (searchMin)
      answer = min(answer, value);
    else
      answer = max(answer, value);
  }
  return answer;
}
```

2 Data Structures

2.1 segmenttree2d

```cpp
struct SegmentTree{
    vector<ll>ST; int N;
    SegmentTree(int Size){
        N = Size; ST.assign(4*N,0);
    }
    void upd(int i, ll v){
        return upd(1,0,N-1,i,v);
    }
    void upd(int n, int l, int r, int i, ll v){
        if(i < l || r < i) return;
        if(l == r){
            ST[n] += v; return;
        }
        upd(2*n,l,(l+r)/2,i,v);
        upd(2*n+1,(l+r)/2+1,r,i,v);
        ST[n] = ST[2*n]+ST[2*n+1];
    }
    ll qry(int i,int j){
        return qry(1,0,N-1,i,j);
    }
    ll qry(int n, int l, int r, int i, int j){
        if(r < i || j < l) return 0;
        if(i <= l && r <= j) return ST[n];
        return query(2*n,l,(l+r)/2,i,j)+query(2*n+1,
(l+r)/2+1,r,i,j);
    }
};
struct SegmenTree2D{
    vector<SegmentTree>ST;
    int N;
    SegmenTree2D(int Size){
        N = Size; ST.resize(4*N,Size);
    }
    void update(int i, int j, int v){
        return update(1,0,N-1,i,j,v);
    }
    void update(int n, int l, int r, int i, int j, ll v){
        if(i < l || r < i) return;
        if(l == r){
            ST[n].update(j,v);
            return;
        }
        update(2*n,l,(l+r)/2,i,j,v);
        update(2*n+1,(l+r)/2+1,r,i,j,v);
        ST[n].update(j,v);
    }
    ll query(int i1, int i2, int j1, int j2){
        return query(1,0,N-1,i1,i2,j1,j2);
    }
    ll query(int n, int l, int r, int i1, int i2, int j1, int
j2){
        if(l > j1 || i1 > r) return 0;
        if(i1 <= l && r <= j1){
            return ST[n].query(i2,j2);
        }
        return query(2*n,l,(l+r)/2,i1,i2,j1,j2)+query(2*n+1,
(l+r)/2+1,r,i1,i2,j1,j2);
```

```
    }
};
```

2.2 sparsetable

```
struct SparseTable{
    vector<vl >SP;
    SparseTable(vl&A){
        int n = A.size();
        SP.push_back(A);
        ll maxlog = 31 - __builtin_clz(n);
        repx(i, 1, maxlog+1){
            vl aux;
            rep(j, n-(1<<i)+1){
                aux.push_back(min(SP[i-1][j], SP[i-1]
[j+(1<<(i-1))]));
            }
            SP.push_back(aux);
        }
    }
    ll op(int l, int r){
        ll maxlog = 31 - __builtin_clz(r-l+1);
        return max(SP[maxlog][l],SP[maxlog][r-(1<<maxlog)+1]);
    }
    ll find(int l, int r, ll m){ // maxi
      ll maxlog = 31 - __builtin_clz(r-l+1);
      for(int i = maxlog; i >= 0; i--){
        if(l + (1<<i) <= r && SP[i][l] < m){
          l += (1<<i);
        }
      }
      return l;
    }
};
```

2.3 dynamic line segment tree

```
struct DynamicLineSegmentTree {
  vector<LineContainer> tree;
  vector<int> L, R;
  ll minX, maxX;
  DynamicLineSegmentTree(ll left, ll right) {
    tree.resize(1);
    L = {0};
    R = {0};
    minX = left;
    maxX = right;
  }
  int newNode() {
    int i = tree.size();
    tree.push_back(LineContainer());
    L.push_back(0);
    R.push_back(0);
    return i;
  }
  void add(int i, ll left, ll right, ll queryLeft, ll
queryRight, ll a, ll b) {
    if (left > queryRight or right < queryLeft)
      return;
    if (queryLeft <= left && right <= queryRight) {
      tree[i].add(a, b);
      return;
    }
    int mid = (left + right) >> 1;
    if (L[i] == 0)
      L[i] = newNode();
    if (R[i] == 0)
      R[i] = newNode();
    add(L[i], left, mid, queryLeft, queryRight, a, b);
    add(R[i], mid + 1, right, queryLeft, queryRight, a, b);
  }
  void add(int left, int right, ll a, ll b) {
    add(0, minX, maxX, left, right, a, b);
  }
  ll query(int i, int left, int right, ll x) {
    ll answer = tree[i].query(x);
    if (left == right)
      return answer;
    int mid = (left + right) >> 1;
    // change max to min for min
    if (x <= mid && L[i] != 0)
      answer = max(answer, query(L[i], left, mid, x));
    if (x > mid && R[i] != 0)
      answer = max(answer, query(R[i], mid + 1, right, x));
    return answer;
  }
  ll query(ll x) { return query(0, minX, maxX, x); }
};
```

2.4 line segment tree

```
struct LineSegmentTree {
  int n;
  vector<LineContainer> tree;
  LineSegmentTree(int n) : n(n) { tree.resize(4 * n); }
  void add(int i, int left, int right, int queryLeft, int
queryRight, ll a,
           ll b) {
    if (left > queryRight or right < queryLeft)
      return;
    if (queryLeft <= left && right <= queryRight) {
      tree[i].add(a, b);
      return;
    }
    int mid = (left + right) >> 1;
    add(i << 1, left, mid, queryLeft, queryRight, a, b);
    add(i << 1 | 1, mid + 1, right, queryLeft, queryRight, a,
b);
  }
  void add(int left, int right, ll a, ll b) {
    add(1, 0, n - 1, left, right, a, b);
  }
  ll query(int i, int left, int right, ll x) {
    ll answer = tree[i].query(x);
    if (left == right)
      return answer;
    int mid = (left + right) >> 1;
    // change max to min for min
    if (x <= mid)
      answer = max(answer, query(i << 1, left, mid, x));
    else
      answer = max(answer, query(i << 1 | 1, mid + 1, right,
x));
    return answer;
  }
  ll query(ll x) { return query(1, 0, n - 1, x); }
};
```

2.5 merge sort tree

```
struct MergeSortTree {
  int n;
  vector<vector<int>> tree;
  vector<int> merge(const vector<int> &left, const vector<int>
&right) {
    vector<int> result(left.size() + right.size());
    int i = 0, j = 0;
    for (int &x : result) {
      if (i == left.size())
        x = right[j++];
      else if (j == right.size())
        x = left[i++];
      else if (left[i] < right[j])
        x = left[i++];
      else
        x = right[j++];
    }
    return result;
  }
  void build(int i, int left, int right, const vector<int>
&values) {
    if (left == right) {
      tree[i] = {values[left]};
      return;
    }
    int mid = (left + right) >> 1;
    build(i << 1, left, mid, values);
    build(i << 1 | 1, mid + 1, right, values);
    tree[i] = merge(tree[i << 1], tree[i << 1 | 1]);
  }
  MergeSortTree(const vector<int> &values) {
    n = values.size();
    tree.resize(n << 2 | 3);
    build(1, 0, n - 1, values);
  }
  int query(int i, int left, int right, int queryLeft, int
queryRight, int min,
            int max) {
    if (left > queryRight or right < queryLeft)
      return 0;
    if (left >= queryLeft and right <= queryRight) {
      auto leftIterator = lower_bound(tree[i].begin(),
tree[i].end(), min);
      auto rightIterator = upper_bound(tree[i].begin(),
tree[i].end(), max);
```

```
      return distance(leftIterator, rightIterator);
    }
    int mid = (left + right) >> 1;
    return query(i << 1, left, mid, queryLeft, queryRight, min,
max) +
           query(i << 1 | 1, mid + 1, right, queryLeft,
queryRight, min, max);
  }
  int query(int left, int right, int min, int max) {
    return query(1, 0, n - 1, left, right, min, max);
  }
};
```

2.6 mo

```
struct Query { int l, r, idx; };
// answer segment queries using only `add(i)`, `remove(i)` and
`get()`
// functions.
//
// nota marcelo: borra el add, remove y get de la funcion mo y
// definelo afuera
//
// complexity: O((N + Q) * sqrt(N) * F)
//   N = length of the full segment
//   Q = amount of queries
//   F = complexity of the `add`, `remove` functions
template <class A, class R, class G, class T>
void mo(vector<Query> &queries, vector<T> &ans, A add, R
remove, G get) {
    int Q = queries.size(), B = (int)sqrt(Q);
    sort(queries.begin(), queries.end(), [&](Query &a, Query
&b) {
        if (p.first / BLOCK_SIZE != q.first / BLOCK_SIZE)
            return p < q;
        return (p.first / BLOCK_SIZE & 1) ? (p.second <
q.second) : (p.second > q.second);
    });
    ans.resize(Q);
    int l = 0, r = 0;
    for (auto &q : queries) {
        while (r < q.r) add(r), r++;
        while (l > q.l) l--, add(l);
        while (r > q.r) r--, remove(r);
        while (l < q.l) remove(l), l++;
        ans[q.idx] = get();
    }
}
```

2.7 ordered set

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template <typename T>
using ordered_set =
    tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;
template <typename T>
using ordered_multiset = tree<T, null_type, less_equal<T>,
rb_tree_tag,

tree_order_statistics_node_update>;
template <typename T> void erase(ordered_multiset<T> &values, T
value) {
  int rank = values.order_of_key(value);
  auto it = values.find_by_order(rank);
  values.erase(it);
}
```

2.8 ordered set

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
typedef tree<int, null_type, less<int>, rb_tree_tag,
             tree_order_statistics_node_update> ordered_set;
// find_by_order(i) -> iterator to ith element
// order_of_key(k) -> position (int) of lower_bound of k
```

2.9 persistent segment tree

```
template <class T, T merge(T, T)> struct PersistentSegmentTree
{
  vector<T> tree;
  vector<int> L, R;
  int n, root = 0;
  PersistentSegmentTree(int n) : n(n) {
    tree.resize(1);
    L = {0};
    R = {0};
  }
  int newNode(T value, int left = 0, int right = 0) {
    int id = tree.size();
    tree.push_back(value);
    L.push_back(left);
    R.push_back(right);
    return id;
  }
  int update(int id, int left, int right, int position, T
value) {
    int newId = newNode(tree[id], L[id], R[id]);
    if (left == right) {
      tree[newId] = value;
      return newId;
    }
    int mid = (left + right) >> 1;
    if (position <= mid)
      L[newId] = update(L[newId], left, mid, position, value);
    else
      R[newId] = update(R[newId], mid + 1, right, position,
value);
    tree[newId] = merge(tree[L[newId]], tree[R[newId]]);
    return newId;
  }
  T query(int id, int left, int right, int qLeft, int qRight) {
    if (left >= qLeft and right <= qRight)
      return tree[id];
    int mid = (left + right) >> 1;
    if (qRight <= mid)
      return query(L[id], left, mid, qLeft, qRight);
    if (qLeft > mid)
      return query(R[id], mid + 1, right, qLeft, qRight);
    return merge(query(L[id], left, mid, qLeft, qRight),
                 query(R[id], mid + 1, right, qLeft, qRight));
  }
  T query(int id, int left, int right) {
    return query(id, 0, n - 1, left, right);
  }
  int update(int id, int position, T value) {
    return root = update(id, 0, n - 1, position, value);
  }
  int update(int position, T value) { return update(root,
position, value); }
};
```

2.10 segment tree lazy

```
template <class T1, class T2, T1 merge(T1, T1),
          void pushUpdate(T2 parent, T2 &child, int, int, int,
int),
          void applyUpdate(T2 update, T1 &node, int, int)>
struct SegmentTreeLazy {
  int n;
  vector<T1> tree;
  vector<T2> lazy;
  vector<bool> isUpdated;
  void build(int i, int left, int right, const vector<T1>
&values) {
    if (left == right) {
      tree[i] = values[left];
      return;
    }
    int mid = (left + right) >> 1;
    build(i << 1, left, mid, values);
    build(i << 1 | 1, mid + 1, right, values);
    tree[i] = merge(tree[i << 1], tree[i << 1 | 1]);
  }
  SegmentTreeLazy(const vector<T1> &values) {
    n = values.size();
    int size = n << 2 | 3;
    tree.resize(size);
    lazy.resize(size);
    isUpdated.resize(size);
    build(1, 0, n - 1, values);
  }
  SegmentTreeLazy() {}
  void push(int i, int left, int right) {
    if (!isUpdated[i])
      return;
    applyUpdate(lazy[i], tree[i], left, right);
    if (left != right) {
      if (isUpdated[i << 1])
        pushUpdate(lazy[i], lazy[i << 1], left, right, left,
                   (left + right) / 2);
      else
```

```
        lazy[i << 1] = lazy[i];
      if (isUpdated[i << 1 | 1])
        pushUpdate(lazy[i], lazy[i << 1 | 1], left, right,
                   (left + right) / 2 + 1, right);
      else
        lazy[i << 1 | 1] = lazy[i];
      isUpdated[i << 1] = 1;
      isUpdated[i << 1 | 1] = 1;
    }
    isUpdated[i] = false;
  }
  void update(int i, int left, int right, int queryLeft, int
queryRight,
              T2 &value) {
    if (left >= queryLeft and right <= queryRight) {
      if (isUpdated[i])
        pushUpdate(value, lazy[i], queryLeft, queryRight, left,
right);
      else
        lazy[i] = value;
      isUpdated[i] = true;
    }
    push(i, left, right);
    if (left > queryRight or right < queryLeft)
      return;
    if (left >= queryLeft and right <= queryRight)
      return;
    update(i << 1, left, (left + right) >> 1, queryLeft,
queryRight, value);
    update(i << 1 | 1, (left + right) / 2 + 1, right,
queryLeft, queryRight,
           value);
    tree[i] = merge(tree[i << 1], tree[i << 1 | 1]);
  }
  void update(int left, int right, T2 value) {
    update(1, 0, n - 1, left, right, value);
  }
  T1 query(int i, int left, int right, int queryLeft, int
queryRight) {
    push(i, left, right);
    if (queryLeft <= left and right <= queryRight)
      return tree[i];
    int mid = (left + right) >> 1;
    if (mid < queryLeft)
      return query(i << 1 | 1, mid + 1, right, queryLeft,
queryRight);
    if (mid >= queryRight)
      return query(i << 1, left, mid, queryLeft, queryRight);
    return merge(query(i << 1, left, mid, queryLeft,
queryRight),
                 query(i << 1 | 1, mid + 1, right, queryLeft,
queryRight));
  }
  T1 query(int left, int right) { return query(1, 0, n - 1,
left, right); }
};
```

2.11 segment tree

```
template <class T, T merge(T, T)> struct SegmentTree {
  int n;
  vector<T> tree;
  SegmentTree() {}
  SegmentTree(vector<T> &values) {
    n = values.size();
    tree.resize(n << 1);
    for (int i = 0; i < n; i++)
      tree[i + n] = values[i];
    for (int i = n - 1; i > 0; i--)
      tree[i] = merge(tree[i << 1], tree[i << 1 | 1]);
  }
  void update(int position, T value) {
    position += n;
    tree[position] = value;
    for (position >>= 1; position > 0; position >>= 1)
      tree[position] = merge(tree[position << 1], tree[position
<< 1 | 1]);
  }
  T query(int left,
          int right) // [left, right]
  {
    bool hasAnswer = false;
    T answer = T();
    for (left += n, right += n + 1; left < right; left >>= 1,
right >>= 1) {
      if (left & 1) {
        answer = hasAnswer ? merge(answer, tree[left]) :
tree[left];
        hasAnswer = true;
        left++;
      }
      if (right & 1) {
        right--;
        answer = hasAnswer ? merge(answer, tree[right]) :
tree[right];
        hasAnswer = true;
      }
    }
    return answer;
  }
};
```

2.12 tuples unordered map

```
using key = tuple<int, int, int>; // define a key type
using bits = bitset<sizeof(key) * 8>;
unordered_map<bits, int> hashmap;
// hashmap[*((long long *)&hash)] = value;
```

2.13 dsu rollback

```
// Complexity: O(alpha n)
struct op {
  int v, u;
  int v_value, u_value;
  op(int _v, int _v_value, int _u, int _u_value)
      : v(_v), v_value(_v_value), u(_u), u_value(_u_value) {}
};
struct union_find_rb {
  vector<int> e;
  stack<op> ops;
  int comps;
  union_find_rb() {}
  union_find_rb(int n) : comps(n) { e.assign(n, -1); }
  int findSet(int x) { return (e[x] < 0 ? x : findSet(e[x])); }
  bool sameSet(int x, int y) { return findSet(x) ==
findSet(y); }
  int size(int x) { return -e[findSet(x)]; }
  bool unionSet(int x, int y) {
    x = findSet(x), y = findSet(y);
    if (x == y)
      return 0;
    if (e[x] > e[y])
      swap(x, y);
    ops.push(op(x, e[x], y, e[y]));
    comps--;
    e[x] += e[y], e[y] = x;
    return 1;
  }
  void rb() {
    if (ops.empty())
      return;
    op last = ops.top();
    ops.pop();
    e[last.v] = last.v_value;
    e[last.u] = last.u_value;
    comps++;
  }
};
```

2.14 dsu bipartite

```
struct Dsu {
    vector<int> p; Dsu(int N = 0) : p(N, -1) {}
    int get(int x) { return p[x] < 0 ? x : get(p[x]); }
    bool sameSet(int a, int b) { return get(a) == get(b); }
    int size(int x) { return -p[get(x)]; }
    vector<vector<int>> S;
    void unite(int x, int y) {
        if ((x = get(x)) == (y = get(y)))
            return S.push_back({-1});
        if (p[x] > p[y]) swap(x, y);
        S.push_back({x, y, p[x], p[y]});
        p[x] += p[y], p[y] = x;
    }
    void rollback() {
        auto a = S.back(); S.pop_back();
        if (a[0] != -1) p[a[0]] = a[2], p[a[1]] = a[3];
    }
};
// check bipartittenes
void make_set(int v) {
    parent[v] = make_pair(v, 0);
    rank[v] = 0;
```

```
    bipartite[v] = true;
}
pair<int, int> find_set(int v) {
    if (v != parent[v].first) {
        int parity = parent[v].second;
        parent[v] = find_set(parent[v].first);
        parent[v].second ^= parity;
    }
    return parent[v];
}
void add_edge(int a, int b) {
    pair<int, int> pa = find_set(a);
    a = pa.first;
    int x = pa.second;
    pair<int, int> pb = find_set(b);
    b = pb.first;
    int y = pb.second;
    if (a == b) {
        if (x == y)
            bipartite[a] = false;
    } else {
        if (rank[a] < rank[b])
            swap (a, b);
        parent[b] = make_pair(a, x^y^1);
        bipartite[a] &= bipartite[b];
        if (rank[a] == rank[b])
            ++rank[a];
    }
}
bool is_bipartite(int v) {
    return bipartite[find_set(v).first];
}
```

2.15 dsu

```
struct DisjointUnionSet {
  int n;
  vector<int> parent, size;
  DisjointUnionSet(int n) : n(n) {
    parent.resize(n);
    size.resize(n);
    for (int i = 0; i < n; i++) {
      parent[i] = i;
      size[i] = 1;
    }
  }
  int get(int u) {
    if (parent[u] == u)
      return u;
    return parent[u] = get(parent[u]);
  }
  int size(int u) { return size[get(u)]; }
  void Union(int u, int v) {
    u = get(u);
    v = get(v);
    if (u == v)
      return;
    if (size[u] < size[v])
      swap(u, v);
    parent[v] = u;
    size[u] += size[v];
  }
  bool sameSet(int u, int v) { return get(u) == get(v); }
};
```

3 Dp

3.1 1d1d

```
#include <bits/stdc++.h>
using namespace std;
#define ll long long
#define ar array
#define rep(i, n) for(int i = 0; i<(int)n; ++i)
#define repx(i, a ,b) for(int i = (int)a; i<(int)b; ++i)
const int mxN = 2e5+5, M = 1e9+7;
int n, s, dp[mxN], t[mxN], f[mxN];
int w(int j, int i){ //
    if(j >= i) return 1e9;  // here is the cost function from (j, i]
    return s*(f[n]-f[j]) + t[i]*(f[i]-f[j]);
}
struct DD{ // index 0 is used as neutral state
    vector<pair<int, int>> v; // (start pos, best k)
    DD() { v.push_back(make_pair(0, 0)); }
    int qry(int i){
        return (--lower_bound(v.begin(), v.end(), make_pair(i+1, 0)))->second;
    }
    void upd(int x){
        for(int i = (int)v.size()-1; i>=0; --i){
            int y = v[i].first, oldk = v[i].second;
            if(y > x && dp[x] + w(x, y) < dp[oldk] + w(oldk, y)) v.pop_back();
            else{
                int l = y+1, r = n+1;
                while(l < r){
                    int mid = (l+r)/2;
                    if(dp[x] + w(x, mid) < dp[oldk] + w(oldk, mid)) r = mid;
                    else l = mid+1;
                }
                if(r != n+1) v.push_back(make_pair(r, x));
break;
            }
        }
        if(v.size() == 0) v.push_back(make_pair(0, x));
    }
};
    cin >> n >> s;
    DD dd;
    t[0] = f[0] = dp[0] = 0;
    for(int i = 1; i<=n; ++i){
        cin >> t[i] >> f[i]; t[i] += t[i-1]; f[i] += f[i-1];
    }
    for(int i = 1; i<=n; ++i){
        int k = dd.qry(i); dp[i] = dp[k] + w(k, i);dd.upd(i);
    }
    cout << dp[n] << '\n';
// https://vjudge.net/problem/OpenJ_Bailian-1180
```

3.2 digitdp

```
int dp[12][12][2]; // dp[i][s][f] {i: posicion, s: estado del problema, f: act < s}
int k;
vector<int> num;
// solve(r) - solve(l-1)
int call(int pos, int cnt, int f){
    if(cnt > k) return 0;
    if(pos == num.size()){
        if(cnt == k) return 1;
        return 0;
    }
    if(dp[pos][cnt][f] != -1) return dp[pos][cnt][f];
    int res = 0, LMT;
    if(f == 0) LMT = num[pos];
    else LMT = 9;
    /// Try to place all the valid digits such that the number doesn't exceed b
    for(int dgt = 0; dgt<=LMT; dgt++){
        int nf = f, ncnt = cnt;
        if(f == 0 && dgt < LMT) nf = 1; /// The number is getting smaller at this position
        if(dgt == d) ncnt++;
        if(ncnt <= k) res += call(pos+1, ncnt, nf);
    }
    return dp[pos][cnt][f] = res;
}
int solve(string s){
    num.clear();
    while(s.size()){
        num.push_back((s.back()-'0')%10);
        s.pop_back();
    }
    reverse(num.begin(), num.end());
    memset(dp, -1, sizeof(dp));
    return call(0, 0, 0);
}
```

3.3 dp divide and conquer

```
// dp(i, j) = min dp(i-1,k-1) + C(k,j) for all k in [0, j]
// C(a,c) + C(b, d) <= C(a,d) + C(b,c) for all a <= b <= c <= d
vector<pair<ll, ll>> c;
vl acum1, acum2;
ll cost(ll i, ll j){
  return c[j].first * (acum1[j+1] - acum1[i]) - (acum2[j+1] - acum2[i]);
}
vector<ll> last, now;
void compute(int l, int r, int optl, int optr){
    if (l > r) return;
    int mid = (l + r) / 2;
```

```
    pair<ll, int> best = {cost(0, mid), -1};
    for(int k = max(1, optl); k < min(mid, optr) + 1; k++)
        best = min(best, {last[k - 1] + cost(k, mid), k});
    now[mid] = best.first;
    compute(l, mid - 1, optl, best.second);
    compute(mid + 1, r, best.second, optr);
}
int main(){
  ios_base::sync_with_stdio(0); cin.tie(0);
  ll n, k, x, w;
  while(cin >> n >> k){
    c.clear();
    for(int i = 0; i < n; i++){
      cin >> x >> w;
      c.push_back({x, w});
    }
    acum1.clear(); acum2.clear();
    acum1.push_back(0); acum2.push_back(0);
    for(int i = 0; i < n; i++){
      acum1.push_back(c[i].second);
      acum2.push_back(c[i].first * c[i].second);
      acum1.back() += acum1[i];
      acum2.back() += acum2[i];
    }
    last.assign(n, INF);
    now.resize(n);
    for(int i = 0; i < k; i++) { compute(0, n - 1, 0, n - 1);
swap(last, now); }
    cout<< last[n-1]<<"\n";
  }
}
```

3.4 knuth

```
int N; vector<int> A;
vector<vector<int>> DP, OPT;
int main(){
    DP.assign(N + 1, vi(N + 1));
    OPT.assign(N + 1, vi(N + 1));
    rep(i, N) DP[i][i + 1] = A[i + 1] - A[i], OPT[i][i + 1] =
i;
    repx(d, 2, N + 1) rep(l, N + 1 - d){
        int r = l + d, l_ = OPT[l][r - 1], r_ = OPT[l + 1][r];
        DP[l][r] = 1e9;
        repx(i, l_, r_ + 1){
            int aux = DP[l][i] + DP[i][r] + A[r] - A[l];
            if (aux < DP[l][r]) DP[l][r] = aux, OPT[l][r] = i;
        }
    }
}
```

3.5 linecontainer

```
struct Line {
    mutable ll a, b, c;
    bool operator<(Line r) const { return a < r.a; }
    bool operator<(ll x) const { return c < x; }
};
// dynamically insert `a*x + b` lines and query for maximum
// at any x all operations have complexity O(log N)
struct LineContainer : multiset<Line, less<>> {
    ll div(ll a, ll b) {
        return a / b - ((a ^ b) < 0 && a % b);
    }
    bool isect(iterator x, iterator y) {
        if (y == end()) return x->c = INF, 0;
        if (x->a == y->a) x->c = x->b > y->b ? INF : -INF;
        else x->c = div(y->b - x->b, x->a - y->a);
        return x->c >= y->c;
    }
    void add(ll a, ll b) {
        // a *= -1, b *= -1 // for min
        auto z = insert({a, b, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y =
erase(y));
        while ((y = x) != begin() && (--x)->c >= y->c) isect(x,
erase(y));
    }
    ll query(ll x) {
        if (empty()) return -INF; // INF for min
        auto l = *lower_bound(x);
        return l.a * x + l.b;
        // return -l.a * x - l.b; // for min
    }
};
```

3.6 sos dp

```
// itera sobre los subconjuntos de todos los subconjuntos en
O(3^n)
for(int mask=0; mask<(1<<n); ++mask){
    for(int smask=mask; smask; smask=(smask-1)&mask){ //
decreasing order
    // for(int s=0;s=s-mask&mask;) // Increasing order
        ;
    }
}
// SOS DP en O(n * 2^n)
for(int mask = 0; mask < (1<<N); ++mask){
  dp[mask][-1] = A[mask]; //handle base case separately (leaf
states)
  for(int i = 0;i < N; ++i){
    if(mask & (1<<i)){
      dp[mask][i] = dp[mask][i-1] + dp[mask^(1<<i)][i-1];
      }else{
      dp[mask][i] = dp[mask][i-1];
      }
    }
  F[mask] = dp[mask][N-1]; // F es la acumulada en todas las
smask de mask
}
```

3.7 broken profile

```
// Common problems solved using DP on broken profile include:
// finding number of ways to fully fill an area (e.g.
chessboard/grid) with some figures (e.g. dominoes)
// finding a way to fill an area with minimum number of figures
// finding a partial fill with minimum number of unfilled space
(or cells, in case of grid)
// finding a partial fill with the minimum number of figures,
such that no more figures can be added
int n, m;
vector<vector<ll>> dp;
void calc (int x = 0, int y = 0, int mask = 0, int next_mask =
0){
    if (x == n) return;
    if (y >= m) dp[x+1][next_mask] += dp[x][mask];
    else{
        int my_mask = 1 << y;
        if(mask & my_mask){
            calc (x, y+1, mask, next_mask);
        }else{
            calc (x, y+1, mask, next_mask | my_mask);
            if (y+1 < m && ! (mask & my_mask) && ! (mask &
(my_mask << 1)))
                calc (x, y+2, mask, next_mask);
        }
    }
}
int main(){
    cin >> n >> m;
    dp.resize (n+1, vector<long long> (1<<m));
    dp[0][0] = 1;
    for (int x=0; x<n; ++x)
        for (int mask=0; mask<(1<<m); ++mask)
            calc (x, 0, mask, 0);
    cout << dp[n][0];
}
```

3.8 lis

```
int find_lis(const vector<int> &a) {
  vector<int> dp;
  for (int i : a) {
    int pos = lower_bound(dp.begin(), dp.end(), i) -
dp.begin();
    if (pos == dp.size())
      dp.push_back(i);
    else
      dp[pos] = i;
  }
  return dp.size();
}
```

4 Geometry

4.1 circlehierarchy

```
int sweep_line_x = -1e8;
const double EPS = 1e-9;
struct Circle { int x, y, r;
    double get_y(bool up_side) const {
        int b = -2 * y;
        ll c = (ll) y * y + (sweep_line_x - x) *
```

```
ll(sweep_line_x - x) - (ll) r * r;
        ll delta = b * (ll) b - 4 * c;
        return (-b + (up_side ? sqrt(delta) : -sqrt(delta))) /
2;
    }
};
struct Curve {
    bool up_side;
    Circle circle;
    bool operator<(const Curve &rhs) const {
        return circle.get_y(up_side) + EPS <
rhs.circle.get_y(rhs.up_side);
    }
};
struct Interval {
    int owner_id;
    Curve up, down;
    bool operator<(const Interval &rhs) const {
        return down < rhs.down || up < rhs.up;
    }
    bool operator==(const Interval &rhs) const {
        return !(*this < rhs) && !(rhs < *this);
    }
};
int n;
Circle circle[maxn];
vector<int> g[maxn];
int main() {
    cin >> n;
    vector<pair<int, int>> events;
    for (int i = 1; i <= n; i++) {
        cin >> circle[i].x >> circle[i].y >> circle[i].r;
        events.emplace_back(circle[i].x - circle[i].r, i);
        events.emplace_back(circle[i].x + circle[i].r, i);
    }
    sort(events.begin(), events.end());
    circle[0] = {0, 0, inf};
    set<Interval> intervals({{0, {true, circle[0]}, {false,
circle[0]}}});
    for (auto tmp : events) {
        int sw_x = tmp.first, circle_id = tmp.second;
        sweep_line_x = sw_x;
        int x = circle[circle_id].x, r = circle[circle_id].r;
        Interval inside = {circle_id, {true,
circle[circle_id]}, {false, circle[circle_id]}};
        if (sw_x == x - r) {
            auto par_it = --intervals.lower_bound(inside);
            auto par_interval = *par_it;
            intervals.erase(par_it);
            g[par_interval.owner_id].push_back(circle_id);
            auto up_interval = par_interval, down_interval =
par_interval;
            up_interval.down = {true, circle[circle_id]};
            down_interval.up = {false, circle[circle_id]};
            intervals.insert(up_interval);
            intervals.insert(down_interval);
            intervals.insert(inside);
        } else {
            auto me = intervals.lower_bound(inside);
            auto joined = *prev(me);
            joined.up = next(me)->up;
            intervals.erase(prev(me));
            intervals.erase(next(me));
            intervals.erase(me);
            intervals.insert(joined);
        }
    }
    for (int i = 0; i <= n; ++i)
        for (auto child : g[i])
            cout << i << ' ' << child << '\n';
}
```

4.2 circle line intersection

```
template <class T>
optional<pair<Point<T>, Point<T>>>
circleLineIntersection(T radius, Point<T> first, Point<T>
second) {
  auto [x1, y1] = first;
  auto [x2, y2] = second;
  T dx = x2 - x1, dy = y2 - y1;
  T a = dy;
  T b = -dx;
  T c = -(a * x1 + b * y1);
  if (c * c > radius * radius * (a * a + b * b)) {
    return nullopt;
  }
  if (c * c == radius * radius * (a * a + b * b)) {
    // TODO: Check if this is correct
    T x0 = -a * c / (a * a + b * b), y0 = -b * c / (a * a + b *
b);
    return make_pair(Point<T>(x0, y0), Point<T>(x0, y0));
  }
  T d = radius * radius - c * c / (a * a + b * b);
  T mult = sqrt(d / (a * a + b * b));
  T x0 = -a * c / (a * a + b * b), y0 = -b * c / (a * a + b *
b);
  T ax = x0 + b * mult, ay = y0 - a * mult;
  T bx = x0 - b * mult, by = y0 + a * mult;
  return make_pair(Point<T>(ax, ay), Point<T>(bx, by));
}
```

4.3 circle and triangles

```
//#define double long long //Para usar enteros
db DEG_to_RAD(db d) { return d * PI / 180.0; }
db RAD_to_DEG(db r) { return r * 180.0 / PI; }
//sweepline rotating a circle around a point
// how many points are in circle radius r
// alpha = atan2(point - center) +- acos(dist/2r)
struct point { db x, y;
    point() { x = y = 0.0; }
    point(db _x, db _y) : x(_x), y(_y) {}
    bool operator <(const point& p) const { return (x < p.x ?
true : (x == p.x && y < p.y)); }
    bool operator == (const point& p) const { return abs(p.x -
x) < EPS && abs(p.y - y) < EPS; }
    point operator + (const point& p) const { return point(x +
p.x, y + p.y); }
    point operator - (const point& p) const { return point(x -
p.x, y - p.y); }
    point operator * (db p) const { return point(x * p, y *
p); }
    point operator / (db p) const { return point(x / p, y /
p); }
    db operator^(const point &p) const {return x * p.y - y *
p.x; }
    db operator*(const point &p) const {return x * p.x + y *
p.y; }
    db norm_sq() const{ return x*x + y*y; }
    point rot(){ return point(-y, x); }
    // by angles but with cross
    bool half() const { return y > 0 || (y == 0 && x > 0); }
    bool operator<(const point &p) const {
        int h1 = half(), h2 = p.half();
        return h1 != h2 ? h1 > h2 : ((*this) ^ p) > 0;
    }
    db ang(){
        double a = atan2(y, x);
        if (a < 0) a += 2.0 * PI;
        return a;
    }
};
ll insideCircle(point p, point c, ll r) { // all integer
version
    ll dx = p.x - c.x, dy = p.y - c.y;
    ll Euc = dx * dx + dy * dy, rSq = r * r;            // all
integer
    return Euc < rSq ? 0 : Euc == rSq ? 1 : 2; } //inside/
border/outside
// P1 an d P2 intersections of circles and radius r -> pos of
centers of circles of intersection
bool circle2PtsRad(point p1, point p2, db r, point &c) {
    db d2 = (p1.x - p2.x) * (p1.x - p2.x) +
                (p1.y - p2.y) * (p1.y - p2.y);
    db det = r * r / d2 - 0.25;
    if (det < 0.0) return false;
    db h = sqrt(det);
    c.x = (p1.x + p2.x) * 0.5 + (p1.y - p2.y) * h;
    c.y = (p1.y + p2.y) * 0.5 + (p2.x - p1.x) * h;
    return true; }          // to get the other center, reverse
p1 and p2
db dist(point& p1, point& p2) {                // Euclidean
distance
    return sqrt((p1.x-p2.x)*(p1.x-p2.x)+ (p1.y-p2.y)*(p1.y-
p2.y)); }
db dist_sq(point p1, point p2) {
    return (p1.x - p2.x)*(p1.x - p2.x)+(p1.y - p2.y)*(p1.y -
p2.y); }
// a = max x, b = max y from the center, AREA
db A_elipse(db a,db b)
{
    return a*b*PI;
```

```
}
// Length of segment with two points on the circunference
// separated by an angle
db chord(db r, db angle)
{
    return sqrt(2*r*r*(1-cos(angle)));
}
//Triangles
db perimeter(db ab, db bc, db ca) {
    return ab + bc + ca; }
db perimeter(point a, point b, point c) {
    return dist(a, b) + dist(b, c) + dist(c, a); }
db area(db ab, db bc, db ca) {
  // Heron's formula, split sqrt(a * b) into sqrt(a) * sqrt(b);
in implementation
    db s = 0.5 * perimeter(ab + bc + ca);
    return sqrt(s) * sqrt(s - ab) * sqrt(s - bc) * sqrt(s -
ca); }
db area(point a, point b, point c) {
    return area(dist(a, b), dist(b, c), dist(c, a)); }
// Area of the circle enclosed by an arc and a chord defined by
an angle
db segment(db r, db angle)
{
    return  angle/2.0*r*r-area(chord(r,angle),r,r);
}
// And overlaping rectangle area > 0
bool rectangles_intersect(point a1,point a2,point b1,point
b2,point& ans1, point& ans2)
{
    if(b1<a1)
    {
        swap(a1,b1);
        swap(a2,b2);
    }
    if(b1.x>=a2.x||b1.y>=a2.y||b2.y<=a1.y)return 0;
    ans1.x=b1.x;
    ans1.y=max(b1.y,a1.y);
    ans2.x=min(b2.x,a2.x);
    ans2.y=min(b2.y,a2.y);
    return 1;
}
struct line { db a, b, c; };
void pointsToLine(point p1, point p2, line &l) {
    if (fabs(p1.x - p2.x) < EPS) {              // vertical
line is fine
        l.a = 1.0;   l.b = 0.0;   l.c = -p1.x;          //
default values
    } else {
        l.a = -(db)(p1.y - p2.y) / (p1.x - p2.x);
        l.b = 1.0;              // IMPORTANT: we fix the value
of b to 1.0
        l.c = -(db)(l.a * p1.x) - p1.y;
} }
bool areParallel(line l1, line l2) {        // check
coefficient a + b
    return (fabs(l1.a-l2.a) < EPS) && (fabs(l1.b-l2.b) <
EPS); }
bool areSame(line l1, line l2) {           // also check
coefficient c
    return areParallel(l1 ,l2) && (fabs(l1.c-l2.c) < EPS); }
// returns true (+ intersection point) if two lines are
intersect
bool areIntersect(line l1, line l2, point &p) {
    if (areParallel(l1, l2)) return false;            // no
intersection
    // solve system of 2 linear algebraic equations with 2
unknowns
    p.x = (l2.b * l1.c - l1.b * l2.c) / (l2.a * l1.b - l1.a *
l2.b);
    // special case: test for vertical line to avoid division
by zero
    if (fabs(l1.b) > EPS) p.y = -(l1.a * p.x + l1.c);
    else                  p.y = -(l2.a * p.x + l2.c);
    return true; }
db rInCircle(db ab, db bc, db ca) {
    return area(ab, bc, ca) / (0.5 * perimeter(ab, bc, ca)); }
db rInCircle(point a, point b, point c) {
    return rInCircle(dist(a, b), dist(b, c), dist(c, a)); }
// assumption: the required points/lines functions have been
written
// returns 1 if there is an inCircle center, returns 0
otherwise
// if this function returns 1, ctr will be the inCircle center
// and r is the same as rInCircle
ll inCircle(point p1, point p2, point p3, point &ctr, db &r) {
    r = rInCircle(p1, p2, p3);
    if (fabs(r) < EPS) return 0;                  // no
inCircle center
    line l1, l2;                    // compute these two angle
bisectors
    db ratio = dist(p1, p2) / dist(p1, p3);
    point p = p2 + (p3 - p2) * (ratio / (1 + ratio));
    pointsToLine(p1, p, l1);
    ratio = dist(p2, p1) / dist(p2, p3);
    p = p1 + (p3 - p1) * (ratio / (1 + ratio));
    pointsToLine(p2, p, l2);
    areIntersect(l1, l2, ctr);          // get their
intersection point
    return 1; }
db rCircumCircle(db ab, db bc, db ca) {
    return ab * bc * ca / (4.0 * area(ab, bc, ca)); }
db rCircumCircle(point a, point b, point c) {
    return rCircumCircle(dist(a, b), dist(b, c), dist(c, a)); }
// assumption: the required points/lines functions have been
written
// returns 1 if there is a circumCenter center, returns 0
otherwise
// if this function returns 1, ctr will be the circumCircle
center
// and r is the same as rCircumCircle
ll circumCircle(point p1, point p2, point p3, point &ctr, db
&r){
    db a = p2.x - p1.x, b = p2.y - p1.y;
    db c = p3.x - p1.x, d = p3.y - p1.y;
    db e = a * (p1.x + p2.x) + b * (p1.y + p2.y);
    db f = c * (p1.x + p3.x) + d * (p1.y + p3.y);
    db g = 2.0 * (a * (p3.y - p2.y) - b * (p3.x - p2.x));
    if (fabs(g) < EPS) return 0;
    ctr.x = (d*e - b*f) / g;
    ctr.y = (a*f - c*e) / g;
    r = dist(p1, ctr);  // r = distance from center to 1 of the
3 points
    return 1; }
// https://www.nayuki.io/res/smallest-enclosing-circle/
computational-geometry-lecture-6.pdf
// O(N) expected time
void smallest_enclosing_circle(vector<point>& pts, point&
center, db& r) {
    random_shuffle(pts.begin(), pts.end());
    center = pts[0]; r = 0;
    ll N = pts.size();
    for(ll i=1;i<N;i++) {
        if (dist(pts[i] , center) > r + EPS) {
            center = pts[i];
            r = 0;
            for(ll j=0;j<i;j++) {
                if (dist(pts[j] , center) > r + EPS) {
                    center = (pts[i] + pts[j]) * 0.5;
                    r = dist(pts[i] , center);
                    for(ll k=0;k<j;k++) {
                        if (dist(pts[k] , center) > r + EPS) {
                            db rr;
                            circumCircle(pts[i], pts[j],
pts[k],center,rr);
                            r = dist(pts[k] , center);
                        }
                    }
                }
            }
        }
    }
}
// returns true if point d is inside the circumCircle defined
by a,b,c
ll inCircumCircle(point a, point b, point c, point d) {
    return (a.x - d.x) * (b.y - d.y) * ((c.x - d.x) * (c.x -
d.x) + (c.y - d.y) * (c.y - d.y)) +
        (a.y - d.y) * ((b.x - d.x) * (b.x - d.x) + (b.y - d.y)
* (b.y - d.y)) * (c.x - d.x) +
        ((a.x - d.x) * (a.x - d.x) + (a.y - d.y) * (a.y - d.y))
* (b.x - d.x) * (c.y - d.y) -
        ((a.x - d.x) * (a.x - d.x) + (a.y - d.y) * (a.y - d.y))
* (b.y - d.y) * (c.x - d.x) -
        (a.y - d.y) * (b.x - d.x) * ((c.x - d.x) * (c.x - d.x)
+ (c.y - d.y) * (c.y - d.y)) -
        (a.x - d.x) * ((b.x - d.x) * (b.x - d.x) + (b.y - d.y)
* (b.y - d.y)) * (c.y - d.y) > 0 ? 1 : 0;
}
bool canFormTriangle(db a, db b, db c) {
    return (a + b > c) && (a + c > b) && (b + c > a); }
```

```
/*
Si un punto tiene un ángulo >= 60, el lado opuesto no es el
menor del triangulo
*/
// Function to find the circle on
// which the given three points lie
// better CIRCUMCENTER
tuple<db, db, db> findCircle(db x1, db y1, db x2, db y2, db x3,
db y3)
{
    db x12 = x1 - x2;
    db x13 = x1 - x3;
    db y12 = y1 - y2;
    db y13 = y1 - y3;
    db y31 = y3 - y1;
    db y21 = y2 - y1;
    db x31 = x3 - x1;
    db x21 = x2 - x1;
    db sx13 = x1*x1 - x3*x3;
    db sy13 = y1*y1 - y3*y3;
    db sx21 = x2*x2 - x1*x1;
    db sy21 = y2*y2 - y1*y1;
    db f = ((sx13) * (x12)
             + (sy13) * (x12)
             + (sx21) * (x13)
             + (sy21) * (x13))
            / (2 * ((y31) * (x12) - (y21) * (x13)));
    db g = ((sx13) * (y12)
             + (sy13) * (y12)
             + (sx21) * (y13)
             + (sy21) * (y13))
            / (2 * ((x31) * (y12) - (x21) * (y13)));
    db c = -x1*x1 - y1*y1 - 2 * g * x1 - 2 * f * y1;
    // eqn of circle be x^2 + y^2 + 2*g*x + 2*f*y + c = 0
    // where centre is (h = -g, k = -f) and radius r
    // as r^2 = h^2 + k^2 - c
    db h = -g;
    db k = -f;
    db sqr_of_r = h * h + k * k - c;
    // r is the radius
    db r = sqrt(sqr_of_r);
    //cout << "Centre = (" << h << ", " << k << ")" << endl;
    //cout << "Radius = " << r;
    return make_tuple(h, k, r);
}
```

4.4 closest points

```
// Marcelo
// sort by x
ll closest(vector<ii> &p) {
  int n = SZ(p);
  set<ii> s;
  ll best = 1e18;
  int j = 0;
  fore(i, 0, n) {
    ll d = ceil(sqrt(best));
    while (p[i].fst - p[j].fst >= best)
      s.erase({p[j].snd, p[j].fst}), j++;
    auto it1 = s.lower_bound({p[i].snd - d, p[i].fst});
    auto it2 = s.upper_bound({p[i].snd + d, p[i].fst});
    for (auto it = it1; it != it2; ++it) {
      ll dx = p[i].fst - it->snd;
      ll dy = p[i].snd - it->fst;
      best = min(best, dx * dx + dy * dy);
    }
    s.insert({p[i].snd, p[i].fst});
  }
  return best;
}
// Enzo
template <class T>
pair<ll, pair<Point<T>, Point<T>>> closest(vector<Point<T>>
points) {
  int n = points.size();
  sort(points.begin(), points.end()); // sort by x
  // sort by y
  auto compare = [](const Point<T> &a, const Point<T> &b) {
    if (a.y != b.y)
      return a.y < b.y;
    return a.x < b.x;
  };
  set<Point<T>, decltype(compare)> uniques(compare);
  ll best = INF;
  pair<Point<T>, Point<T>> answer;
  int j = 0;
  for (int i = 0; i < n; i++) {
    while (j < i && points[i].x - points[j].x >= best)
      uniques.erase(points[j++]);
    ll distance = sqrt(best) + 1;
    // [y_i - d, y_i + d]
    auto start = uniques.lower_bound(Point<T>(-INF, points[i].y
- distance));
    auto end = uniques.upper_bound(Point<T>(INF, points[i].y +
distance));
    for (auto it = start; it != end; it++) {
      ll current = points[i].squaredDistance(*it);
      if (best <= current)
        continue;
      best = current;
      answer = {points[i], *it};
    }
    uniques.insert(points[i]);
  }
  return {best, answer};
}
```

4.5 convexhull

```
vector<Point> convexHull(vector<Point> points) {
  if (points.size() < 3)
    return points;
  sort(points.begin(), points.end());
  vector<Point> hull;
  // Lower hull
  for (Point &point : points) {
    // change <= to < for collineal
    while (hull.size() >= 2 &&
           point.left(hull[hull.size() - 2], hull.back()) < 0)
      hull.pop_back();
    hull.push_back(point);
  }
  hull.pop_back();
  int m = hull.size();
  // Upper hull
  for (int i = points.size() - 1; i >= 0; --i) {
    Point &point = points[i];
    // change <= to < for collineal
    while (hull.size() >= m + 2 &&
           point.left(hull[hull.size() - 2], hull.back()) < 0)
      hull.pop_back();
    hull.push_back(point);
  }
  hull.pop_back();
  return hull;
}
```

4.6 delaunay

```
// Returns planar graph representing Delaunay's triangulation.
// Edges for each vertex are in ccw order.
// It can work with doubles, but also integers (replace long
double in line 51)
typedef struct QuadEdge* Q;
struct QuadEdge {
  int id,used;
  pt o;
  Q rot,nxt;
  QuadEdge(int id=-1, pt
o=pt(INF,INF)):id(id),used(0),o(o),rot(0),nxt(0){}
  Q rev(){return rot->rot;}
  Q next(){return nxt;}
  Q prev(){return rot->next()->rot;}
  pt dest(){return rev()->o;}
};
Q edge(pt a, pt b, int ida, int idb){
  Q e1=new QuadEdge(ida,a);
  Q e2=new QuadEdge(idb,b);
  Q e3=new QuadEdge;
  Q e4=new QuadEdge;
  tie(e1->rot,e2->rot,e3->rot,e4->rot)={e3,e4,e2,e1};
  tie(e1->nxt,e2->nxt,e3->nxt,e4->nxt)={e1,e2,e4,e3};
  return e1;
}
void splice(Q a, Q b){
  swap(a->nxt->rot->nxt,b->nxt->rot->nxt);
  swap(a->nxt,b->nxt);
}
void del_edge(Q& e, Q ne){
  splice(e,e->prev()); splice(e->rev(),e->rev()->prev());
  delete e->rev()->rot; delete e->rev();
  delete e->rot; delete e;
  e=ne;
}
```

```
Q conn(Q a, Q b){
  Q e=edge(a->dest(),b->o,a->rev()->id,b->id);
  splice(e,a->rev()->prev());
  splice(e->rev(),b);
  return e;
}
auto area(pt p, pt q, pt r){return (q-p)%(r-q);}
// is p in circunference formed by (a,b,c)?
bool in_c(pt a, pt b, pt c, pt p){
  // Warning: this number is O(max_coord^4).
  // Consider using int128 or using an alternative method for
this function
  long double p2=p*p,A=a*a-p2,B=b*b-p2,C=c*c-p2;
  return area(p,a,b)*C+area(p,b,c)*A+area(p,c,a)*B>EPS;
}
pair<Q,Q> build_tr(vector<pt>& p, int l, int r){
  if(r-l+1<=3){
    Q a=edge(p[l],p[l+1],l,l+1),b=edge(p[l+1],p[r],l+1,r);
    if(r-l+1==2) return {a,a->rev()};
    splice(a->rev(),b);
    auto ar=area(p[l],p[l+1],p[r]);
    Q c=abs(ar)>EPS?conn(b,a):0;
    if(ar>=-EPS) return {a,b->rev()};
    return {c->rev(),c};
  }
  int m=(l+r)/2;
  auto [la,ra]=build_tr(p,l,m);
  auto [lb,rb]=build_tr(p,m+1,r);
  while(1){
    if(ra->dest().left(lb->o,ra->o)) ra=ra->rev()->prev();
    else if(lb->dest().left(lb->o,ra->o)) lb=lb->rev()->next();
    else break;
  }
  Q b=conn(lb->rev(),ra);
  auto valid=[&](Q e){return b->o.left(e->dest(),b->dest());};
  if(ra->o==la->o) la=b->rev();
  if(lb->o==rb->o) rb=b;
  while(1){
    Q L=b->rev()->next();
    if(valid(L)) while(in_c(b->dest(),b->o,L->dest(),L->next()-
>dest())) del_edge(L,L->next());
    Q R=b->prev();
    if(valid(R)) while(in_c(b->dest(),b->o,R->dest(),R->prev()-
>dest())) del_edge(R,R->prev());
    if(!valid(L)&&!valid(R)) break;
    if(!valid(L)||(valid(R)&&in_c(L->dest(),L->o,R->o,R-
>dest()))) b=conn(R,b->rev());
    else b=conn(b->rev(),L->rev());
  }
  return {la,rb};
}
vector<vector<int>> delaunay(vector<pt> v){
  int n=SZ(v); auto tmp=v;
  vector<int> id(n); iota(ALL(id),0);
  sort(ALL(id),[&](int l, int r){return v[l]<v[r];});
  fore(i,0,n) v[i]=tmp[id[i]];
  assert(unique(ALL(v))==v.end());
  vector<vector<int>> g(n);
  int col=1;
  fore(i,2,n) col&=abs(area(v[i],v[i-1],v[i-2]))<=EPS;
  if(col){
    fore(i,1,n) g[id[i-1]].pb(id[i]),g[id[i]].pb(id[i-1]);
  }
  else{
    Q e=build_tr(v,0,n-1).fst;
    vector<Q> edg={e};
    for(int i=0;i<SZ(edg);e=edg[i++]){
      for(Q at=e;!at->used;at=at->next()){
        at->used=1;
        g[id[at->id]].pb(id[at->rev()->id]);
        edg.pb(at->rev());
      }
    }
  }
  return g;
}
```

4.7 halfplane intersection

```
// obtain the convex polygon that results from intersecting the
given list
// of halfplanes, represented as lines that allow their left
side
// assumes the halfplane intersection is bounded
template <class T>
vector<Point<T>> halfPlaneIntersection(vector<Line<T>> lines) {
  // add box if needed
  lines.push_back(Line(Point(-INF, -INF), Point(INF, -INF)));
  lines.push_back(Line(Point(INF, -INF), Point(INF, INF)));
  lines.push_back(Line(Point(INF, INF), Point(-INF, INF)));
  lines.push_back(Line(Point(-INF, INF), Point(-INF, -INF)));
  sort(lines.begin(), lines.end());
  // q: start, h: end
  int n = lines.size(), q = 1, h = 0;
  // just a silly deque
  vector<Line<T>> c(n + 2);
  for (int i = 0; i < n; i++) {
    while (q < h && lines[i].out(c[h].getPoint(c[h - 1])))
      h--;
    while (q < h && lines[i].out(c[q].getPoint(c[q + 1])))
      q++;
    c[++h] = lines[i];
    // check if parallel
    // If not using integers consider using EPS
    if (q < h && abs((c[h].b - c[h].a).cross(c[h - 1].b - c[h -
1].a)) == 0) {
      // if parallel are oposite, returns null answer
      if ((c[h].b - c[h].a).dot(c[h - 1].b - c[h - 1].a) <= 0)
        return {};
      h--;
      if (lines[i].out(c[h].a))
        c[h] = lines[i];
    }
  }
  while (q < h - 1 && c[q].out(c[h].getPoint(c[h - 1])))
    h--;
  while (q < h - 1 && c[h].out(c[q].getPoint(c[q + 1])))
    q++;
  // No intersection
  if (h - q <= 1)
    return {};
  vector<Point<T>> polygon;
  c[h + 1] = c[q];
  for (int i = q; i <= h; i++)
    polygon.push_back(c[i].getPoint(c[i + 1]));
  return polygon;
}
```

4.8 minkowski

```
// MARCELO
void reorder_polygon(vector<P> &ps) {
  int pos = 0;
  repx(i, 1, (int)ps.size()) {
    if (ps[i].y < ps[pos].y || (ps[i].y == ps[pos].y && ps[i].x
< ps[pos].x))
      pos = i;
  }
  rotate(ps.begin(), ps.begin() + pos, ps.end());
}
vector<P> minkowski(vector<P> ps, vector<P> qs) {
  // the first vertex must be the lowest
  reorder_polygon(ps);
  reorder_polygon(qs);
  ps.push_back(ps[0]);
  ps.push_back(ps[1]);
  qs.push_back(qs[0]);
  qs.push_back(qs[1]);
  vector<P> result;
  int i = 0, j = 0;
  while (i < ps.size() - 2 || j < qs.size() - 2) {
    result.push_back(ps[i] + qs[j]);
    auto z = (ps[i + 1] - ps[i]) % (qs[j + 1] - qs[j]);
    if (z >= 0 && i < ps.size() - 2)
      ++i;
    if (z <= 0 && j < qs.size() - 2)
      ++j;
  }
  return result;
}
// ENZO
bool reverseCompare(Point &a, Point &b) {
  if (a.y == b.y)
    return a.x < b.x;
  return a.y < b.y;
};
void reorder(vector<Point> &points) {
  int index = 0;
  for (int i = 1; i < (int)points.size(); i++)
    if (reverseCompare(points[i], points[index]))
      index = i;
  rotate(points.begin(), points.begin() + index, points.end());
}
```

```cpp
// O(n + m)
// Returns the Minkowski sum of two polygons a and b
// Both polygons must be given in counter-clockwise order
vector<Point> minkowski(vector<Point> a, vector<Point> b) {
  reorder(a);
  a.push_back(a[0]);
  a.push_back(a[1]);
  reorder(b);
  b.push_back(b[0]);
  b.push_back(b[1]);
  vector<Point> result;
  int i = 0, j = 0;
  int n = (int)a.size(), m = (int)b.size();
  while (i < n - 2 || j < m - 2) {
    result.push_back(a[i] + b[j]);
    auto cross = (a[i + 1] - a[i]).cross(b[j + 1] - b[j]);
    if (i < n - 2 && cross >= 0)
      i++;
    if (j < m - 2 && cross <= 0)
      j++;
  }
  return result;
}
// Example: to compute the Minkowski difference of A and B, use
// minkowski(A, transform(B, [](Point point) { return Point(-
point.x, -point.y);
// }));
vector<Point> transform(const vector<Point> &points,
                        function<Point(Point)> callback) {
  int n = (int)points.size();
  vector<Point> result(n);
  for (int i = 0; i < n; i++)
    result[i] = callback(points[i]);
  return result;
}
```

4.9 point in polygon

```cpp
// 0: outside, 1: inside, 2: boundary
template <class T>
int pointInPolygon(vector<Point<T>> &polygon, Point<T> point) {
  int count = 0;
  bool isInBoundary = false;
  int n = polygon.size();
  for (int i = 0; i < n; i++) {
    Point<T> pointA = polygon[i], pointB = polygon[(i + 1) %
n];
    T cross = (pointB - pointA).cross(point - pointA);
    if (cross == 0 && point.x >= min(pointA.x, pointB.x) &&
        point.x <= max(pointA.x, pointB.x) &&
        point.y >= min(pointA.y, pointB.y) &&
        point.y <= max(pointA.y, pointB.y))
      return 2;
    if ((point.y <= pointA.y) == (point.y <= pointB.y))
      continue;
    T left = point.x * (pointB.y - pointA.y);
    T right = pointA.x * (pointB.y - pointA.y) +
              (point.y - pointA.y) * (pointB.x - pointA.x);
    if ((pointB.y - pointA.y) < 0)
      left *= -1, right *= -1;
    count += left < right;
  }
  if (isInBoundary)
    return 2;
  if (count % 2 == 0)
    return 0;
  return 1;
}
```

4.10 point

```cpp
template <class T> struct Point {
  T x, y;
  const T EPS = 0;
  Point() {}
  Point(T x, T y) : x(x), y(y) {}
  friend istream &operator>>(istream &is, Point &point) {
    is >> point.x >> point.y;
    return is;
  }
  bool operator=(const Point &point) const { return Point(x,
y); }
  bool operator==(const Point &point) const {
    return point.x == x && point.y == y;
  }
  bool operator!=(const Point &point) const { return !(point ==
*this); }
  bool operator<(const Point &point) const {
    if (x == point.x)
      return y < point.y;
    return x < point.x;
  }
  bool operator>(const Point &point) const {
    return !(*this > point) && (*this != point);
  }
  bool operator>=(const Point &point) {
    return *this > point || *this == point;
  }
  bool operator<=(const Point &point) {
    return *this < point || *this == point;
  }
  Point operator+(const Point &point) const {
    return Point(x + point.x, y + point.y);
  }
  Point operator-(const Point &point) const {
    return Point(x - point.x, y - point.y);
  }
  Point operator+(T value) const { return Point(x + value, y +
value); }
  Point operator-(T value) const { return Point(x - value, y -
value); }
  Point operator*(T v) const { return Point(x * v, y * v); }
  Point operator/(T v) const { return Point(x / v, y / v); }
  T cross(const Point &point) const { return x * point.y - y *
point.x; }
  T cross(const Point &a, const Point &b) const {
    return (a - *this).cross(b - *this);
  }
  int left(const Point &point) const {
    T crossProduct = this->cross(point);
    if (abs(crossProduct) < EPS)
      return 0;
    if (crossProduct > EPS)
      return 1;
    return -1;
  }
  int left(const Point &a, const Point &b) const {
    T crossProduct = this->cross(a, b);
    if (abs(crossProduct) < EPS)
      return 0;
    if (crossProduct > EPS)
      return 1;
    return -1;
  }
  T squaredLength() const { return x * x + y * y; }
  T squaredDistance(const Point &point) {
    T deltaX = point.x - x, deltaY = point.y - y;
    return deltaX * deltaX + deltaY * deltaY;
  }
  T distance(const Point &point) { return
sqrt(squaredDistance(point)); }
  void rotate(T radians) {
    T newX = x * cos(radians) - y * sin(radians),
      newY = x * sin(radians) + y * cos(radians);
    x = newX, y = newY;
  };
  double angle() { return atan2(y, x); }
  T dot(const Point &other) const { return x * other.x + y *
other.y; }
};
```

4.11 polygon

```cpp
template <class T> struct Polygon {
  vector<Point<T>> points, relative;
  int n;
  int sign(T value) {
    if (value == 0)
      return 0;
    return value > 0 ? 1 : -1;
  }
  Polygon(const int n) {
    this->n = n;
    points.resize(n);
  }
  Polygon(const vector<Point<T>> &points) : points(points),
n(points.size()) {}
  friend istream &operator>>(istream &is, Polygon &polygon) {
    for (auto &point : polygon.points)
      is >> point;
    return is;
  }
  T doubleArea() {
    T sum = 0;
```

```
    for (int i = 0; i < n; i++)
      sum += points[i].cross(points[(i + 1) % n]);
    return abs(sum);
  }
  T area() { return doubleArea() / 2; }
  T boundaryLatticePoints() {
    T count = 0;
    for (int i = 0; i < n; i++) {
      Point<T> delta = points[i] - points[(i + 1) % n];
      count += gcd(abs(delta.x), abs(delta.y));
    }
    return count;
  }
  T insideLatticePoints() {
    return (doubleArea() - boundaryLatticePoints()) / 2 + 1;
  }
  T latticePoints() { return boundaryLatticePoints() +
insideLatticePoints(); }
};
```

4.12 segment

```
inline int sign(ll x) {
  if (x == 0)
    return 0;
  return x > 0 ? 1 : -1;
}
inline bool intersect(ll a, ll b, ll c, ll d) {
  if (a > b)
    swap(a, b);
  if (c > d)
    swap(c, d);
  return max(a, c) <= min(b, d);
}
template <class T> struct Segment {
  Point<T> a, b;
  Segment() {}
  Segment(Point<T> a, Point<T> b) : a(a), b(b) {}
  friend istream &operator>>(istream &is, Segment &segment) {
    is >> segment.a >> segment.b;
    return is;
  }
  // Highly tested
  bool intersects(const Segment &other) const {
    Point<T> c = other.a, d = other.b;
    if (c.cross(a, d) == 0 && c.cross(b, d) == 0)
      return intersect(a.x, b.x, c.x, d.x) && intersect(a.y,
b.y, c.y, d.y);
    return sign(a.cross(b, c)) != sign(a.cross(b, d)) &&
           sign(c.cross(d, a)) != sign(c.cross(d, b));
  }
  // Not tested
  pair<bool, pair<Point<T>, Point<T>>>
  getIntersection(const Segment &other) const {
    if (!intersects(other))
      return {false, {{0, 0}, {0, 0}}};
    Point<T> c = other.a, d = other.b;
    Point<T> ab = b - a, cd = d - c, ac = c - a;
    T det = ab.cross(cd);
    // If not using integers consider using EPS
    if (det == 0) {
      Point<T> leftA = min(a, b), rightA = max(a, b);
      Point<T> leftB = min(c, d), rightB = max(c, d);
      Point<T> start = max(leftA, leftB);
      Point<T> end = min(rightA, rightB);
      return {true, {start, end}};
    }
    // This maybe fail using integers, consider using long
double
    T t = ac.cross(cd) / det;
    Point<T> intersectionPoint(a.x + (b.x - a.x) * t, a.y +
(b.y - a.y) * t);
    return {true, {intersectionPoint, intersectionPoint}};
  }
};
```

4.13 spherical distance

```
//  * Description: Returns the shortest distance on the sphere
with radius radius between the points
//  * with azimuthal angles (longitude) f1 ($\phi_1$) and f2 ($
\phi_2$) from x axis and zenith angles
//  * (latitude) t1 ($\theta_1$) and t2 ($\theta_2$) from z
axis (0 = north pole). All angles measured
//  * in radians. The algorithm starts by converting the
spherical coordinates to cartesian coordinates
//  * so if that is what you have you can use only the two last
rows. dx*radius is then the difference
//  * between the two points in the x direction and d*radius is
the total distance between the points.
//  * Status: tested on kattis:airlinehub
double sphericalDistance(double f1, double t1, double f2,
double t2, double radius) {
  double dx = sin(t2)*cos(f2) - sin(t1)*cos(f1);
  double dy = sin(t2)*sin(f2) - sin(t1)*sin(f1);
  double dz = cos(t2) - cos(t1);
  double d = sqrt(dx*dx + dy*dy + dz*dz);
  return radius*2*asin(d/2);
}
```

4.14 point in triangle

```
// Check if point is in Triangle abc
// IMPORTANT: Points on the edge are considered inside
// IMPORTANT: Check for overflow (int128 may be needed)
// int128 needs custom abs function
bool pointInTriangle(Point a, Point b, Point c, Point point) {
  auto area = abs(a.cross(b, c));
  auto candidateArea =
      abs(point.cross(a, b)) + abs(point.cross(b, c)) +
abs(point.cross(c, a));
  return area == candidateArea;
}
```

4.15 point in convex polygon

```
// Construction: O(n), Query: O(log n)
// Only works for convex polygons
struct PointInPolygon {
  vector<Point> points, sequence;
  Point translation;
  int n, m;
  PointInPolygon(vector<Point> points) : points(points) {
    n = (int)points.size();
    // order (x, y)
    rotate(points.begin(), min_element(points.begin(),
points.end()),
           points.end());
    translation = points[0];
    m = n - 1;
    sequence.resize(m);
    for (int i = 0; i < m; i++)
      sequence[i] = points[i + 1] - points[0];
  }
  bool query(Point point) {
    point = point - translation;
    if (sequence.front().cross(point) == 0)
      return sequence.front().dot(point) >= 0 &&
             sequence.front().squaredLength() >=
                 point.squaredLength(); // Use >= to consider
points on edge as
                                        // inside, < to
consider them as outside
    if (sequence.back().cross(point) == 0)
      return sequence.back().dot(point) >= 0 &&
             sequence.back().squaredLength() >=
                 point.squaredLength(); // Use >= to consider
points on edge as
                                        // inside, < to
consider them as outside
    if (sequence.front().cross(point) < 0)
      return false;
    if (sequence.back().cross(point) > 0)
      return false;
    int left = 0, right = m - 1;
    while (right - left > 1) {
      int mid = (left + right) / 2;
      if (sequence[mid].cross(point) >= 0)
        left = mid;
      else
        right = mid;
    }
    // This dont check if point is on the edge of the polygon,
modify
    // pointInTriangle if needed
    return pointInTriangle(sequence[left], sequence[(left + 1)
% m],
                           Point(0, 0), point);
  }
};
```

4.16 line

```
template <class T> struct Line {
  Point<T> a, b;
  double angle;
```

```
  Line() {}
  Line(Point<T> a, Point<T> b) : a(a), b(b), angle((b -
a).angle()) {}
  bool operator<(const Line &other) const { return angle <
other.angle; }
  pair<int, pair<Point<T>, Point<T>>> intersect(const Line
&other) const {
    Point<T> ab = b - a;
    Point<T> cd = other.b - other.a;
    Point<T> ac = other.a - a;
    T det = ab.cross(cd);
    if (abs(det) <= EPS) {
      // Same line
      if (ab.cross(other.a - a) == 0)
        return {2, {a, b}};
      // Paralels
      return {0, {Point<T>(0, 0), Point<T>(0, 0)}};
    }
    // Unique intersection
    // This maybe fail using integers, consider using long
double
    T t = ac.cross(cd) / det;
    Point<T> point(a.x + ab.x * t, a.y + ab.y * t);
    return {1, {point, point}};
  }
  // For Half-plane-intersection
  Point<T> getPoint(const Line &other) const {
    Point<T> ab = b - a, cd = other.b - other.a, ac = other.a -
a;
    T det = ab.cross(cd);
    return Point<T>(a.x + ab.x * ac.cross(cd) / det,
                    a.y + ab.y * ac.cross(cd) / det);
  }
  // Check if point is at right of the line
  bool out(const Point<T> &point) const { return point.left(a,
b) < 0; }
};
```

5 Graphs

5.1 2sat

```
// assign bolean values to variables who are given in CNF (a v
b) ^ (not a v b)
struct TwoSatSolver {
    int n_vars;
    int n_vertices;
    vector<vector<int>> adj, adj_t;
    vector<bool> used;
    vector<int> order, comp;
    vector<bool> assignment;
    TwoSatSolver(int _n_vars) : n_vars(_n_vars), n_vertices(2 *
n_vars), adj(n_vertices), adj_t(n_vertices), used(n_vertices),
order(), comp(n_vertices, -1), assignment(n_vars){
        order.reserve(n_vertices);
    }
    void dfs1(int v) {
        used[v] = true;
        for (int u : adj[v]) {
            if (!used[u])
                dfs1(u);
        }
        order.push_back(v);
    }
    void dfs2(int v, int cl) {
        comp[v] = cl;
        for (int u : adj_t[v]) {
            if (comp[u] == -1)
                dfs2(u, cl);
        }
    }
    bool solve_2SAT() {
        order.clear();
        used.assign(n_vertices, false);
        for (int i = 0; i < n_vertices; ++i) {
            if (!used[i])
                dfs1(i);
        }
        comp.assign(n_vertices, -1);
        for (int i = 0, j = 0; i < n_vertices; ++i) {
            int v = order[n_vertices - i - 1];
            if (comp[v] == -1)
                dfs2(v, j++);
        }
        assignment.assign(n_vars, false);
        for (int i = 0; i < n_vertices; i += 2) {
            if (comp[i] == comp[i + 1])
                return false;
            assignment[i / 2] = comp[i] > comp[i + 1];
        }
        return true;
    }
    void add_disjunction(int a, bool na, int b, bool nb) {
        // na and nb signify whether a and b are to be negated
        a = 2 * a ^ na;
        b = 2 * b ^ nb;
        int neg_a = a ^ 1;
        int neg_b = b ^ 1;
        adj[neg_a].push_back(b);
        adj[neg_b].push_back(a);
        adj_t[b].push_back(neg_a);
        adj_t[a].push_back(neg_b);
    }
    static void example_usage() {
        TwoSatSolver solver(3); // a, b, c
        solver.add_disjunction(0, false, 1, true);  //     a  v
not b
        solver.add_disjunction(0, true, 1, true);   // not a  v
not b
        solver.add_disjunction(1, false, 2, false); //     b  v
c
        solver.add_disjunction(0, false, 0, false); //     a  v
a
        assert(solver.solve_2SAT() == true);
        auto expected = vector<bool>(True, False, True);
        assert(solver.assignment == expected);
    }
};
```

5.2 chuliu

```
//O(n*m) minimum spanning tree in directed graph
//returns -1 if not possible
//included i-th edge if take[i]!=0
typedef int tw; tw INF=1ll<<30;
struct edge{int u,v,id;tw len;};
struct ChuLiu{
    int n; vector<edge> e;
    vector<int> inc,dec,take,pre,num,id,vis;
    vector<tw> inw;
    void add_edge(int x, int y, tw w){
        inc.pb(0); dec.pb(0); take.pb(0);
        e.pb({x,y,SZ(e),w});
    }
    ChuLiu(int n):n(n),pre(n),num(n),id(n),vis(n),inw(n){}
    tw doit(int root){
        auto e2=e;
        tw ans=0; int eg=SZ(e)-1,pos=SZ(e)-1;
        while(1){
            fore(i,0,n) inw[i]=INF,id[i]=vis[i]=-1;
            for(auto ed:e2) if(ed.len<inw[ed.v]){
                inw[ed.v]=ed.len; pre[ed.v]=ed.u;
                num[ed.v]=ed.id;
            }
            inw[root]=0;
            fore(i,0,n) if(inw[i]==INF) return -1;
            int tot=-1;
            fore(i,0,n){
                ans+=inw[i];
                if(i!=root)take[num[i]]++;
                int j=i;
                while(vis[j]!=i&&j!
=root&&id[j]<0)vis[j]=i,j=pre[j];
                if(j!=root&&id[j]<0){
                    id[j]=++tot;
                    for(int k=pre[j];k!=j;k=pre[k]) id[k]=tot;
                }
            }
            if(tot<0)break;
            fore(i,0,n) if(id[i]<0)id[i]=++tot;
            n=tot+1; int j=0;
            fore(i,0,SZ(e2)){
                int v=e2[i].v;
                e2[j].v=id[e2[i].v];
                e2[j].u=id[e2[i].u];
                if(e2[j].v!=e2[j].u){
                    e2[j].len=e2[i].len-inw[v];
                    inc.pb(e2[i].id);
                    dec.pb(num[v]);
                    take.pb(0);
                    e2[j++].id=++pos;
                }
```

```
            }
            e2.resize(j);
            root=id[root];
        }
        while(pos>eg){
            if(take[pos]>0) take[inc[pos]]++, take[dec[pos]]--;
            pos--;
        }
        return ans;
    }
};
```

5.3 dinic

```
// time: O(E V^2)
//      O(E V^(2/3)) / O(E sqrt(E))    unit capacities
//      O(E sqrt(V))   (hopcroft-karp) unit networks
// unit network: c in {0,1} & forall v, indeg<=1 or outdeg<=1
// min-cut: nodes reachable from s in final residual graph
// en grafo bipartito:
// max matching = max flow
// min edge cover = conjunto minimo de arista que cubran todos los nodos
// min edge cover = n - max_matching
// min vertex cover = conjunto minimo de nodos que cubran todas las aristas
// min vertex cover = max_matching
// maximal independent set = conjunto de vertices que no compartan aristas
// maximal independent set = min edge cover
// dilworth: pongo todos los nodos a la izquierda y una copia a la derecha
// se une un nodo x de la izquierda a y de la derecha si existe camino de x a y
// anticadena = conjunto de nodos tal que no hay camino entre ellos
// cadena = conjunto de nodos tal que de cada uno puedo llegar al siguiente
// anticadena de mayor tamaño = minima cantidad de anticadenas que cubren todos los nodos
// min chain cover = conseguimos el max matching, si existe x -> y, y -> z, en el matching
//        consideramos x -> y -> z como una sola cadena (con dsu o un vector posiblemente)
// Anticadena maxima = hacemos min vertex cover y tomamos los nodos que no tienen ninguna
//                     copia en el cubrimiento, este es el tamaño maximo
// Flujo con demandas (no necesariamente el maximo)
// Agregar s' y t' nuevos source and sink
// c'(s', v) = sum(d(u, v) for u in V) \forall arista (s', v)
// c'(v, t') = sum(d(v, w) for w in V) \forall arista (v, t')
// c'(u, v) = c(u, v) - d(u, v) \forall aristas antiguas
// c'(t, s) = INF (el flujo por esta arista es el flujo real)*/
class Dinic{
    struct Edge { int to, rev; ll f, c; };
    int n, t_; vector<vector<Edge>> G;
    vl D; vi q, W;
    bool bfs(int s, int t){
        W.assign(n, 0); D.assign(n, -1); D[s] = 0;
        int f = 0, l = 0; q[l++] = s;
        while (f < l){
            int u = q[f++];
            for (const Edge &e : G[u]) if (D[e.to] == -1 && e.f < e.c)
                D[e.to] = D[u] + 1, q[l++] = e.to;
        }
        return D[t] != -1;
    }
    ll dfs(int u, ll f){
        if (u == t_) return f;
        for (int &i = W[u]; i < (int)G[u].size(); ++i){
            Edge &e = G[u][i]; int v = e.to;
            if (e.c <= e.f || D[v] != D[u] + 1) continue;
            ll df = dfs(v, min(f, e.c - e.f));
            if (df > 0) { e.f += df, G[v][e.rev].f -= df; return df; }
        }
        return 0;
    }
public:
    Dinic(int N) : n(N), G(N), D(N), q(N) {}
    void addEdge(int u, int v, ll cap){
        G[u].push_back({v, (int)G[v].size(), 0, cap});
        G[v].push_back({u, (int)G[u].size() - 1, 0, 0}); // cap if bidirectional
    }
    ll maxFlow(int s, int t){
        t_ = t; ll ans = 0;
        while (bfs(s, t)) while (ll dl = dfs(s, LLONG_MAX)) ans += dl;
        return ans;
    }
};
```

5.4 dominatortree

```
//idom[i]=parent of i in dominator tree with root=rt, or -1 if not exists
int n,rnk[MAXN],pre[MAXN],anc[MAXN],idom[MAXN],semi[MAXN],low[MAXN];
vector<int> g[MAXN],rev[MAXN],dom[MAXN],ord;
void dfspre(int pos){
    rnk[pos]=SZ(ord); ord.pb(pos);
    for(auto x:g[pos]){
        rev[x].pb(pos);
        if(rnk[x]==n) pre[x]=pos,dfspre(x);
    }
}
int eval(int v){
    if(anc[v]<n&&anc[anc[v]]<n){
        int x=eval(anc[v]);
        if(rnk[semi[low[v]]]>rnk[semi[x]]) low[v]=x;
        anc[v]=anc[anc[v]];
    }
    return low[v];
}
void dominators(int rt){
    fore(i,0,n){
        dom[i].clear(); rev[i].clear();
        rnk[i]=pre[i]=anc[i]=idom[i]=n;
        semi[i]=low[i]=i;
    }
    ord.clear(); dfspre(rt);
    for(int i=SZ(ord)-1;i;i--){
        int w=ord[i];
        for(int v:rev[w]){
            int u=eval(v);
            if(rnk[semi[w]]>rnk[semi[u]])semi[w]=semi[u];
        }
        dom[semi[w]].pb(w); anc[w]=pre[w];
        for(int v:dom[pre[w]]){
            int u=eval(v);
            idom[v]=(rnk[pre[w]]>rnk[semi[u]]?u:pre[w]);
        }
        dom[pre[w]].clear();
    }
    for(int w:ord) if(w!=rt&&idom[w]!=semi[w]) idom[w]=idom[idom[w]];
    fore(i,0,n) if(idom[i]==n)idom[i]=-1;
}
```

5.5 floydwarshall

```
// calculate distances between every pair of nodes in O(V^3) time.
// works with negative edges, but not negative cycles.
void floyd(const vector<vector<pair<ll, int>>> &G, vector<vector<ll>> &D) {
    int N = G.size();
    D.assign(N, vector<ll>(N, INF));
    rep(u, N) D[u][u] = 0;
    rep(u, N) for (auto [w, v] : G[u]) D[u][v] = w;
    rep(k, N) rep(u, N) rep(v, N)
        D[u][v] = min(D[u][v], D[u][k] + D[k][v]);
}
```

5.6 hungarian

```
// Minimum/Maximum cost of a perfect matching in complete graph
// O(n^3)
template<class T>
class Hungarian{
    T inf = numeric_limits<T>::max() / 2;
    bool maxi, swapped = false;
    vector<vector<T>> cost;
    vector<T> u, v;
    vl p, way;
    int l, r;
public:
    // left/right == partition sizes
    Hungarian(int left, int right, bool maximizing){
        l = left, r = right, maxi = maximizing;
        if (swapped = l > r) swap(l, r);
        cost.assign(l + 1, vector<T>(r + 1, 0));
```

```
        u.assign(l + 1, 0); v.assign(r + 1, 0);
        p.assign(r + 1, 0); way.assign(r + 1, 0);
    }
    void add_edge(int l, int r, T w){
        assert(l and r);  // indices start from 1 !!
        if (swapped) swap(l, r);
        cost[l][r] = maxi ? -w : w;
    }
    // execute after all edges were added
    void calculate(){
        repx(i, 1, l + 1){
            vector<bool> used(r+1, false);
            vector<T> minv(r+1, inf);
            int j0 = 0; p[0] = i;
            while (p[j0]){
                int j1, i0 = p[j0]; used[j0] = true;
                T delta = inf;
                repx(j, 1, r + 1) if (not used[j])
                {
                    T cur = cost[i0][j] - u[i0] - v[j];
                    if (cur < minv[j]) minv[j] = cur, way[j] =
j0;

                    if(minv[j] < delta) delta = minv[j], j1 =
j;

                }
                rep(j, r + 1){
                    if (used[j]) u[p[j]] += delta, v[j] -=
delta;

                    else minv[j] -= delta;
                }
                j0 = j1;
            }
            while (j0) p[j0] = p[way[j0]], j0 = way[j0];
        }
    }
    // execute after executing calculate()
    T answer() { return maxi ? v[0] : -v[0]; }
    bool are_matched(int l, int r){
        if (swapped) swap(l, r);
        return p[r] == l;
    }
};
int main(){
    ios_base::sync_with_stdio(0); cin.tie(0);
    ll n;
    cin >> n;
    ll d[n][n], pos = (n+1)/2;
    Hungarian<ll> h(pos, pos, 0);
    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            cin >> d[i][j];
    for(int i = 0; i < n; i += 2){
        for(int j = 0; j < n; j+= 2){
            ll cost = 0;
            if(j > 0) cost += d[i][j-1];
            if(j < n-1) cost += d[i][j+1];
            h.add_edge(i/2+1, j/2+1, cost);
        }
    }
    h.calculate();
    cout << h.answer() << "\n";
}
```

5.7 mincostmaxflow

```
template <class T>
class MCMF{
    typedef pair<T, T> pTT;
    T INF = numeric_limits<T>::max();
    struct Edge{
        int v; T c, w;
        Edge(int v, T c, T w) : v(v), c(c), w(w) {}
    };
    int n; vvi E;
    vector<Edge> L; vi F; vector<T> D, P; vector<bool> V;
    bool dij(int s, int t){
        D.assign(n, INF); F.assign(n, -1); V.assign(n, false);
        // D[s] = 0;
        // rep(_, n){
        //     int best = -1;
        //     rep(i, n) if (!V[i] && (best == -1 || D[best] >
D[i])) best = i;
        //     if (D[best] >= INF) break;
        //     V[best] = true;
        //     for (int e : E[best]){
        //         Edge ed = L[e];
        //         if (ed.c == 0) continue;
        //         T toD = D[best] + ed.w + P[best] - P[ed.v];
        //         if (toD < D[ed.v]) D[ed.v] = toD, F[ed.v] =
e;
        //     }
        // }
        priority_queue<pair<T, int>> q;
        D[s] = 0;
        q.push({D[s], s});
        while(!q.empty()) {
            int x = q.top().second;
            ll d = -q.top().first;
            q.pop();
            if(D[x] != d) continue;
            for(int e: E[x]) {
                Edge ed = L[e];
                if(ed.c == 0) continue;
                T toD = D[x] + ed.w + P[x] - P[ed.v];
                if(D[ed.v] > toD) {
                    D[ed.v] = toD;
                    q.push({-D[ed.v], ed.v});
                    F[ed.v] = e;
                }
            }
        }
        return D[t] < INF;
    }
    pTT augment(int s, int t){
        pTT flow(L[F[t]].c, 0);
        for (int v = t; v != s; v = L[F[v] ^ 1].v)
            flow.ff = min(flow.ff, L[F[v]].c), flow.ss +=
L[F[v]].w;
        for (int v = t; v != s; v = L[F[v] ^ 1].v)
            L[F[v]].c -= flow.ff, L[F[v] ^ 1].c += flow.ff;
        return flow;
    }
public:
    MCMF(int n) : n(n), E(n), D(n), P(n, 0), V(n, 0) {}
    pTT mcmf(int s, int t){
        pTT ans(0, 0);
        if (!dij(s, t)) return ans;
        rep(i, n) if (D[i] < INF) P[i] += D[i];
        while (dij(s, t)){
            auto flow = augment(s, t);
            ans.ff += flow.ff, ans.ss += flow.ff * flow.ss;
            rep(i, n) if (D[i] < INF) P[i] += D[i];
        }
        return ans;
    }
    void addEdge(int u, int v, T c, T w){
        E[u].pb(L.size()); L.eb(v, c, w);
        E[v].pb(L.size()); L.eb(u, 0, -w);
    }
};
```

5.8 partiallypersistentdsu

```
struct PPDSU {
  vector<vector<pair<int, int>>> par;
  int time = 0; //initial time
  PPDSU(int n) : par(n + 1, {{-1, 0}}) {}
  bool merge(int u, int v) {
    ++time;
    if ((u = root(u, time)) == (v = root(v, time))) return 0;
    if (par[u].back().first > par[v].back().first) swap(u, v);
    par[u].push_back({par[u].back().first +
par[v].back().first, time});
    par[v].push_back({u, time}); //par[v] = u
    return 1;
  }
  bool same(int u, int v, int t) {
    return root(u, t) == root(v, t);
  }
  int root(int u, int t) { //root of u at time t
    if (par[u].back().first >= 0 && par[u].back().second <= t)
return root(par[u].back().first, t);
    return u;
  }
  int size(int u, int t) { //size of the component of u at time
t
    u = root(u, t);
    int l = 0, r = (int) par[u].size() - 1, ans = 0;
    while (l <= r) {
      int mid = l + r >> 1;
      if (par[u][mid].second <= t) ans = mid, l = mid + 1;
      else r = mid - 1;
    }
```

```
    return -par[u][ans].first;
  }
};
```

5.9 scc

```
const int mxN = 2e5+5;
int n, m, who[mxN], cc = 0;
vector<int> adj[mxN], adj_rev[mxN], adj2[mxN];
bool used[mxN];
vector<int> order, component, root_nodes;
void dfs1(int v) {
    used[v] = true;
    for (auto u : adj[v])
        if (!used[u])
            dfs1(u);
    order.push_back(v);
}
void dfs2(int v) {
    used[v] = true;
    component.push_back(v);
    for (auto u : adj_rev[v]){
        if (!used[u]){
            dfs2(u);
        }
    }
}
int main(){
    std::ios_base::sync_with_stdio(false); std::cin.tie(0);
std::cout.tie(0);
  cin >> n >> m;
  for(int i = 0; i<m; ++i){
    int a, b;
    cin >> a >> b, --a, --b;
    adj[a].push_back(b);
    adj_rev[b].push_back(a);
  }
  for(int i = 0; i<n; ++i){
    if(!used[i]){
      dfs1(i);
    }
  }
  memset(used, false, sizeof(used));
  reverse(order.begin(), order.end());
  for(int v : order){
    if(!used[v]){
      dfs2(v);
      int root = component.front();
      for(int u : component){
        who[u] = root;
      }
      root_nodes.push_back(root);
      component.clear();
      cc++;
    }
  }
    // condensation graph construction
  for(int v = 0; v<n; ++v){
    for(auto u : adj[v]){
      int root_v = who[v];
            int root_u = who[u];
      if (root_u != root_v){
        adj2[root_v].push_back(root_u);
      }
    }
  }
}
```

5.10 specialpathwithcentroids

```
int n, k, sz[mxN], mxtree[mxN], ans;
vector<int> adj[mxN];
bool used[mxN];
int dfs(int u, int p = -1){
    sz[u] = 1;
    for(int v : adj[u]){
        if(v != p && !used[v]){
            sz[u] += dfs(v, u);
        }
    }
    return sz[u];
}
int get_centroid(int u, int ms, int p = -1){ // u = node, ms =
size of the current tree delimited by previous centroids, p =
parent
    for(int v : adj[u]){
        if(v != p && !used[v]){
            if(sz[v]*2 > ms) return get_centroid(v, ms, u);
        }
    }
    return u;
}
void solve(int u){
    int sz_curr_tree = dfs(u);
    int centroid = get_centroid(u, sz_curr_tree);
    used[centroid] = 1;
    // do something with the centroid
    for(int v : adj[centroid]){
    }
    // end that something with the centroid
    // solve for child trees centroids
    for(int v : adj[centroid]){
        if(used[v]) continue;
        solve(v);
    }
}
```

5.11 stoer wagner

```
const int N = 155;
// O(n^3) but faster, 1 indexed, Finds global minimum cut
// Finds edge connectivity: minimum number of edges to be
removed such that the graph gets disconnected
// vertex connectivity: minimun number of vertices to be
removed such that the graph gets disconnected
// for vertex connec this has no use, but you can iterate over
(s, t), split each node x s.t x is not s or t
// in (x1, x2) with an edge of capactity 1 and replace all
edges (u, v) into (u2, v1), (v2, u1) with cap 1
mt19937
rnd(chrono::steady_clock::now().time_since_epoch().count());
struct StoerWagner {
  int n;
  long long G[N][N], dis[N];
  int idx[N];
  bool vis[N];
  const long long inf = 1e18;
  StoerWagner() {}
  StoerWagner(int _n) {
    n = _n;
    memset(G, 0, sizeof G);
  }
  void add_edge(int u, int v, long long w) { //undirected edge,
multiple edges are merged into one edge
    if (u != v) {
      G[u][v] += w;
      G[v][u] += w;
    }
  }
  long long solve() {
    long long ans = inf;
    for (int i = 0; i < n; ++ i) idx[i] = i + 1;
    shuffle(idx, idx + n, rnd);
    while (n > 1) {
      int t = 1, s = 0;
      for (int i = 1; i < n; ++ i) {
        dis[idx[i]] = G[idx[0]][idx[i]];
        if (dis[idx[i]] > dis[idx[t]]) t = i;
      }
      memset(vis, 0, sizeof vis);
      vis[idx[0]] = true;
      for (int i = 1; i < n; ++ i) {
        if (i == n - 1) {
          if (ans > dis[idx[t]]) ans = dis[idx[t]]; //idx[s] -
idx[t] is in two halves of the  mincut
          if (ans == 0) return 0;
          for (int j = 0; j < n; ++ j) {
            G[idx[s]][idx[j]] += G[idx[j]][idx[t]];
            G[idx[j]][idx[s]] += G[idx[j]][idx[t]];
          }
          idx[t] = idx[-- n];
        }
        vis[idx[t]] = true;
        s = t;
        t = -1;
        for (int j = 1; j < n; ++ j) {
          if (!vis[idx[j]]) {
            dis[idx[j]] += G[idx[s]][idx[j]];
            if (t == -1 || dis[idx[t]] < dis[idx[j]]) t = j;
          }
        }
      }
    }
    return ans;
```

```
  }
};
int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  int t, cs = 0;
  cin >> t;
  while (t--) {
    int n, m;
    cin >> n >> m;
    StoerWagner st(n);
    while (m--) {
      int u, v, w;
      cin >> u >> v >> w;
      st.add_edge(u, v, w);
    }
    cout << "Case #" << ++cs << ": " << st.solve() << '\n';
  }
  return 0;
}
```

5.12 bridges

```
void is_bridge(int v, int to); // some function to process the
found bridge
int n;                          // number of nodes
vector<vector<int>> adj;        // adjacency list of graph
vector<bool> visited;
vector<int> tin, low;
int timer;
void dfs(int v, int p = -1) {
  visited[v] = true;
  tin[v] = low[v] = timer++;
  bool parent_skipped = false;
  for (int to : adj[v]) {
    if (to == p && !parent_skipped) {
      parent_skipped = true;
      continue;
    }
    if (visited[to]) {
      low[v] = min(low[v], tin[to]);
    } else {
      dfs(to, v);
      low[v] = min(low[v], low[to]);
      if (low[to] > tin[v])
        is_bridge(v, to);
    }
  }
}
void find_bridges() {
  timer = 0;
  visited.assign(n, false);
  tin.assign(n, -1);
  low.assign(n, -1);
  for (int i = 0; i < n; ++i) {
    if (!visited[i])
      dfs(i);
  }
}
```

5.13 cut points

```
int n;                    // number of nodes
vector<vector<int>> adj; // adjacency list of graph
vector<bool> visited;
vector<int> tin, low;
int timer;
void dfs(int v, int p = -1) {
  visited[v] = true;
  tin[v] = low[v] = timer++;
  int children = 0;
  for (int to : adj[v]) {
    if (to == p)
      continue;
    if (visited[to]) {
      low[v] = min(low[v], tin[to]);
    } else {
      dfs(to, v);
      low[v] = min(low[v], low[to]);
      if (low[to] >= tin[v] && p != -1)
        is_cutpoint(v); // change
      ++children;
    }
  }
  if (p == -1 && children > 1)
    is_cutpoint(v); // change
}
void find_cutpoints() {
  timer = 0;
  visited.assign(n, false);
  tin.assign(n, -1);
  low.assign(n, -1);
  for (int i = 0; i < n; ++i) {
    if (!visited[i])
      dfs(i);
  }
}
```

5.14 heavylightdecomposition

```
template <class ST, class T, T merge(T a, T b)>
struct HeavyLightDecomposition
{
  vector<vector<int>> tree;
  vector<int> parent, depth, heavy, head, position, size;
  int currentPosition;
  ST segmentTree;
  HeavyLightDecomposition() {}
  HeavyLightDecomposition(vector<vector<int>> &tree) :
tree(tree)
  {
    int n = tree.size();
    parent.resize(n);
    depth.resize(n);
    heavy.assign(n, -1);
    head.resize(n);
    position.resize(n);
    size.resize(n);
    currentPosition = 0;
    dfs(0);
    decompose(0, 0);
  }
  int dfs(int node)
  {
    size[node] = 1;
    int maxSubtreeSize = 0;
    for (int child : tree[node])
    {
      if (child == parent[node])
        continue;
      parent[child] = node;
      depth[child] = depth[node] + 1;
      int subtreeSize = dfs(child);
      size[node] += subtreeSize;
      if (subtreeSize <= maxSubtreeSize)
        continue;
      maxSubtreeSize = subtreeSize;
      heavy[node] = child;
    }
    return size[node];
  }
  void decompose(int node, int nodeHead)
  {
    head[node] = nodeHead;
    position[node] = currentPosition++;
    if (heavy[node] != -1)
      decompose(heavy[node], nodeHead);
    for (int child : tree[node])
      if (child != parent[node] && child != heavy[node])
        decompose(child, child);
  }
  int query(int a, int b)
  {
    T answer;
    bool hasValue = false;
    while (head[a] != head[b])
    {
      if (depth[head[a]] > depth[head[b]])
        swap(a, b);
      T current = segmentTree.query(position[head[b]],
position[b]);
      answer = hasValue ? merge(answer, current) : current;
      hasValue = true;
      b = parent[head[b]];
    }
    if (depth[a] > depth[b])
      swap(a, b);
    T last = segmentTree.query(position[a], position[b]);
    return hasValue ? merge(answer, last) : last;
  }
  vector<T> sorted(vector<T> &values)
  {
    int n = values.size();
```

```
    vector<T> result(n);
    for (int i = 0; i < n; i++)
      result[position[i]] = values[i];
    return result;
  }
  void update(int node, T value)
{ segmentTree.update(position[node], value); }
  // void update(int subtree, T value) {
  //   segmentTree.update(position[subtree], position[subtree]
+ size[subtree] -
  //   1,
  //                      value);
  // }
};
// HeavyLightDecomposition<SegmentTree<int, merge>, int, merge>
heavyLight(tree);
// heavyLight.segmentTree = SegmentTree<int,
merge>(heavyLight.sorted(values));
```

5.15 fast chu liu

```
// Minimun spanning tree for directed graph
// Complexity: O(E log V)
template <class T> struct fast_chu_liu {
  struct edge {
    int u, v;
    T len;
  };
  struct node {
    edge key;
    node *l = nullptr, *r = nullptr;
    T delta = 0;
    node(edge e) : key(e) {}
    void prop() {
      key.len += delta;
      if (l)
        l->delta += delta;
      if (r)
        r->delta += delta;
      delta = 0;
    }
    edge top() {
      prop();
      return key;
    }
  };
  int n;
  vector<edge> e;
  fast_chu_liu(int n) : n(n) {}
  void add_edge(int x, int y, T w) { e.push_back({x, y, w}); }
  node *merge(node *a, node *b) {
    if (!a || !b)
      return a ? a : b;
    a->prop(), b->prop();
    if (a->key.len > b->key.len)
      swap(a, b);
    swap(a->l, (a->r = merge(b, a->r)));
    return a;
  }
  void pop(node *&a) {
    a->prop();
    a = merge(a->l, a->r);
  }
  // returns (-1, emptyset) if not reachable
  pair<T, vector<int>> solve(int root) {
    union_find_rb uf(n);
    vector<node *> heap(n, nullptr);
    for (edge &ed : e)
      heap[ed.v] = merge(heap[ed.v], new node(ed));
    T ans = 0;
    vector<int> seen(n, -1), path(n), par(n);
    seen[root] = root;
    vector<edge> Q(n), in(n, {-1, -1, 0});
    deque<tuple<int, int, vector<edge>>> cycs;
    for (int s = 0; s < n; s++) {
      int u = s, qi = 0, w;
      while (seen[u] < 0) {
        if (!heap[u])
          return {-1, {}};
        edge ed = heap[u]->top();
        heap[u]->delta -= ed.len, pop(heap[u]);
        Q[qi] = ed, path[qi++] = u, seen[u] = s;
        ans += ed.len, u = uf.findSet(ed.u);
        if (seen[u] == s) {
          node *cyc = nullptr;
          int end = qi, t = (int)uf.ops.size();
          do
            cyc = merge(cyc, heap[w = path[--qi]]);
          while (uf.unionSet(u, w));
          u = uf.findSet(u), heap[u] = cyc, seen[u] = -1;
          cycs.push_front({u, t, {&Q[qi], &Q[end]}});
        }
      }
      for (int i = 0; i < qi; i++)
        in[uf.findSet(Q[i].v)] = Q[i];
    }
    for (auto &[u, t, comp] : cycs) {
      while ((int)uf.ops.size() > t)
        uf.rb();
      edge inEdge = in[u];
      for (auto &ed : comp)
        in[uf.findSet(ed.v)] = ed;
      in[uf.findSet(inEdge.v)] = inEdge;
    }
    for (int i = 0; i < n; i++)
      par[i] = in[i].u;
    return {ans, par};
  }
};
```

6 Math

6.1 128bits

```
#if defined(__LP64__) || defined(_WIN64)
using int128 = __int128;
using uint128 = __uint128_t;
#else
using int128 = ll;
using uint128 = ull;
#endif
template <typename T>
typename enable_if<is_same<T, int128>::value || is_same<T,
uint128>::value,
                   istream &>::type
operator>>(istream &is, T &value) {
  string input;
  is >> input;
  value = 0;
  for (char character : input)
    if (isdigit(character))
      value = 10 * value + character - '0';
  if (input[0] == '-')
    value *= -1;
  return is;
}
template <typename T>
typename enable_if<is_same<T, int128>::value || is_same<T,
uint128>::value,
                   ostream &>::type
operator<<(ostream &os, T value) {
  string output;
  bool isNegative = value < 0;
  if (isNegative) {
    value *= -1;
    output += '-';
  }
  do {
    output += value % 10 + '0';
    value /= 10;
  } while (value > 0);
  reverse(output.begin() + isNegative, output.end());
  return os << output;
}
```

6.2 linear diophantine

```
ii extendedEuclid(ll a, ll b){
    ll x, y; //a*x + b*y = gcd(a,b)
    if (b == 0) return {1, 0};
    auto p = extendedEuclid(b, a%b);
    x = p.second;
    y = p.first - (a/b)*x;
    if(a*x + b*y == -__gcd(a,b)) x=-x, y=-y;
    return {x, y};
}
pair<ii, ii> diophantine(ll a, ll b, ll r){
    //a*x+b*y=r where r is multiple of gcd(a,b);
    ll d = __gcd(a, b);
    a/=d; b/=d; r/=d;
    auto p = extendedEuclid(a, b);
    p.first*=r; p.second*=r;
```

```
    assert(a*p.first + b*p.second == r);
    return {p, {-b, a}}; //solutions: p+t*ans.second
}
```

6.3 combinatorics

```
const int mxN = 1e6+5, M = 1e9+7;
ll f1[mxN], f2[mxN], iv[mxN];
ll binpow(ll a, ll b){
    ll res = 1;
    while(b){
        if(b & 1) res = res*a%M;
        a = a*a%M;
        b >>= 1;
    }
    return res;
}
ll modinv(ll x){
    return binpow(x, M-2);
}
ll nCr(int a, int b){
    return f1[a]*f2[b]%M*f2[a-b]%M;
    // // si b muy chico y probablemente a muy grande
    // if(b > a) return 0;
    // ll res = 1;
    // for(ll i = 0; i<b; ++i){
    //     ans *= (n-i)%M;
    //     ans %= M; ans += M; ans %= M;
    // }
    // for(ll i = 0; i<b; ++i){
    //     ans *= (modinv(i+1)) % M;
    //     ans %= M; ans += M; ans %= M;
    // }
    // return ans;
}
void solve(){
    int a, b;
    cin >> a >> b;
    cout << nCr(a, b) << "\n";
}
signed main(){
    iv[1] = 1;
    for(int i = 2; i<mxN; ++i){
        iv[i]=M-M/i*iv[M%i]%M;
    }
    f1[0]=f2[0]=1;
    for(int i = 1; i<mxN;++i){
        f1[i]=f1[i-1]*i%M;
        f2[i]=f2[i-1]*iv[i]%M;
    }
    solve();
}
```

6.4 crt

```
pair<ll, ll> solve_crt(const vector<pair<ll, ll>> &eqs) {
    ll a0 = eqs[0].first, p0 = eqs[0].second;
    repx(i, 1, eqs.size()) {
        ll a1 = eqs[i].first, p1 = eqs[i].second;
        ll k1, k0;
        ll d = ext_gcd(p1, p0, k1, k0);
        a0 -= a1;
        if (a0 % d != 0) return {-1, -1};
        p0 = p0 / d * p1;
        a0 = a0 / d * k1 % p0 * p1 % p0 + a1;
        a0 = (a0 % p0 + p0) % p0;
    }
    return {a0, p0};
}
```

6.5 fft

```
#define M_PIl 3.1415926535897932384626433832795028841L
typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C> &a)
{
    int n = a.size(), L = 31 - __builtin_clz(n);
    static vector<complex<long double>> R(2, 1);
    static vector<C> rt(2, 1);
    for (static int k = 2; k < n; k *= 2)
    {
        R.resize(n);
        rt.resize(n);
        auto x = polar(1.0L, M_PIl / k);
        repx(i, k, 2 * k) rt[i] = R[i] = i & 1 ? R[i / 2] * x :
R[i / 2];
    }
    vector<int> rev(n);
    rep(i, n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i, n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2) for (int i = 0; i < n; i +=
2 * k) rep(j, k)
    {
        auto x = (double *)&rt[j + k], y = (double *)&a[i + j +
k];
        C z(x[0] * y[0] - x[1] * y[1], x[0] * y[1] + x[1] *
y[0]);
        a[i + j + k] = a[i + j] - z;
        a[i + j] += z;
    }
}
vd conv(const vl &a, const vl &b)
{
    if (a.empty() || b.empty()) return {};
    vd res(a.size() + b.size() - 1);
    int L = 32 - __builtin_clz(res.size()), n = 1 << L;
    vector<C> in(n), out(n);
    copy(a.begin(), a.end(), in.begin());
    rep(i, b.size()) in[i].imag(b[i]);
    fft(in);
    for (auto &x : in) x *= x;
    rep(i, n) out[i] = in[-i & (n - 1)] - conj(in[i]);
    fft(out);
    rep(i, res.size()) res[i] = imag(out[i]) / (4 * n);
    return res;
}
```

```
//slower
vl convMod(const vl &a, const vl &b, int M)
{
    if (a.empty() || b.empty()) return {};
    vl res(a.size() + b.size() - 1);
    int B = 32 - __builtin_clz(res.size()), n = 1 << B, cut =
int(sqrt(M));
    vector<C> L(n), R(n), outs(n), outl(n);
    rep(i, a.size()) L[i] = C((int)a[i] / cut, (int)a[i] %
cut);
    rep(i, b.size()) R[i] = C((int)b[i] / cut, (int)b[i] %
cut);
    fft(L), fft(R);
    rep(i, n)
    {
        int j = -i & (n - 1);
        outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
        outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
    }
    fft(outl), fft(outs);
    rep(i, res.size())
    {
        ll av = ll(real(outl[i]) + .5), cv = ll(imag(outs[i])
+ .5);
        ll bv = ll(imag(outl[i]) + .5) + ll(real(outs[i])
+ .5);
        res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
    }
    return res;
}
```

6.6 fht

```
ll c1[MAXN+9],c2[MAXN+9];//MAXN must be power of 2!
void fht(ll* p, int n, bool inv){
    for(int l=1;2*l<=n;l*=2)for(int i=0;i<n;i+=2*l)fore(j,0,l){
        ll u=p[i+j],v=p[i+l+j];
        if(!inv)p[i+j]=u+v,p[i+l+j]=u-v; // XOR
        else p[i+j]=(u+v)/2,p[i+l+j]=(u-v)/2;
        //if(!inv)p[i+j]=v,p[i+l+j]=u+v; // AND
        //else p[i+j]=-u+v,p[i+l+j]=u;
        //if(!inv)p[i+j]=u+v,p[i+l+j]=u; // OR
        //else p[i+j]=v,p[i+l+j]=u-v;
    }
}
// like polynomial multiplication, but XORing exponents
// instead of adding them (also ANDing, ORing)
vector<ll> multiply(vector<ll>& p1, vector<ll>& p2){
    int n=1<<(32-__builtin_clz(max(SZ(p1),SZ(p2))-1));
    fore(i,0,n)c1[i]=0,c2[i]=0;
    fore(i,0,SZ(p1))c1[i]=p1[i];
    fore(i,0,SZ(p2))c2[i]=p2[i];
    fht(c1,n,false);fht(c2,n,false);
    fore(i,0,n)c1[i]*=c2[i];
    fht(c1,n,true);
    return vector<ll>(c1,c1+n);
}
```

6.7 gauss mod 2

```
vector<vector<int>>
solveHomogeneousSystemMod2(vector<vector<int>> &A,
                                                int maxSolutions
= 1e5) {
  int rows = A.size(), columns = A[0].size();
  int variables = columns - 1;
  vector<int> pivotPosition(rows, -1);
  int pivotRow = 0;
  for (int pivotColumn = 0; pivotColumn < variables && pivotRow
< rows;
       pivotColumn++) {
    int pivotIndex = pivotRow;
    while (pivotIndex < rows && A[pivotIndex][pivotColumn] ==
0)
      pivotIndex++;
    if (pivotIndex == rows)
      continue;
    swap(A[pivotRow], A[pivotIndex]);
    pivotPosition[pivotRow] = pivotColumn;
    for (int i = 0; i < rows; i++)
      if (i != pivotRow && A[i][pivotColumn] & 1)
        for (int j = pivotColumn; j < columns; j++)
          A[i][j] = A[i][j] ^ A[pivotRow][j];
    pivotRow++;
  }
  vector<bool> isPivot(variables);
  for (int position : pivotPosition) {
    if (position == -1)
      continue;
    isPivot[position] = true;
  }
  vector<int> freeColumns;
  for (int j = 0; j < variables; j++)
    if (!isPivot[j])
      freeColumns.push_back(j);
  if (freeColumns.empty())
    return {vector<int>(variables)};
  vector<vector<int>> solutions;
  for (int freeColumn : freeColumns) {
    vector<int> solution(variables);
    solution[freeColumn] = 1;
    for (int i = 0; i < rows; i++) {
      if (pivotPosition[i] == -1)
        continue;
      int positionColumn = pivotPosition[i];
      solution[positionColumn] = A[i][freeColumn];
    }
    solutions.push_back(solution);
    if (solutions.size() == maxSolutions)
      break;
  }
  return solutions;
}
```

6.8 gauss

```
const double EPS = 1e-9;
// solve a system of equations.
// complexity: O(min(N, M) * N * M)
// `a` is a list of rows
// the last value in each row is the result of the equation
// return values:
//  0 -> no solutions
//  1 -> unique solution, stored in `ans`
// -1 -> infinitely many solutions, one of which is stored in
`ans`
// UNTESTED
int gauss(vector<vector<double>> a, vector<double> &ans) {
    int N = a.size(), M = a[0].size() - 1;
    vector<int> where(M, -1);
    for (int j = 0, i = 0; j < M && i < N; j++) {
        int sel = i;
        repx(k, i, N) if (abs(a[k][j]) > abs(a[sel][j])) sel =
k;
        if (abs(a[sel][j]) < EPS) continue;
        repx(k, j, M + 1) swap(a[sel][k], a[i][k]);
        where[j] = i;
        rep(k, N) if (k != i) {
            double c = a[k][j] / a[i][j];
            repx(l, j, M + 1) a[k][l] -= a[i][l] * c;
        }
        i++;
    }
    ans.assign(M, 0);
    rep(i, M) if (where[i] != -1) ans[i] = a[where[i]][M] /
a[where[i]][i];
    rep(i, N) {
        double sum = 0;
        rep(j, M) sum += ans[j] * a[i][j];
        if (abs(sum - a[i][M]) > EPS) return 0;
    }
    rep(i, M) if (where[i] == -1) return -1;
    return 1;
}
```

6.9 gcd lcm

```
#define gcd __gcd
template <typename T>
typename enable_if<is_integral<T>::value, T>::type lcm(T a, T
b) {
  return a / gcd(a, b) * b;
}
```

6.10 matrix

```
#define mat vector<vector<ll>>
const int M =1e9+7;
const ll M2 = (ll)M*M;
mat cn(int n, int m){
    return vector<vector<ll>>(n, vector<ll>(m));
}
mat mult(mat &a, mat &b){
    mat c = cn(a.size(), b[0].size());
    for(int i=0; i<a.size(); ++i){
        for(int k = 0; k<b.size(); ++k){
            for(int j = 0; j<b[0].size(); ++j){
                c[i][j]+=a[i][k]*b[k][j];
                if(c[i][j]>=M2){
                    c[i][j] -= M2;
                }
            }
        }
    }
    for(int i = 0; i<a.size(); ++i){
        for(int j=0; j<b[0].size(); ++j){
            c[i][j] %= M;
        }
    }
    return c;
}
```

6.11 mobius

```
short mu[MAXN] = {0,1};
void mobius(){
    repx(i,1,MAXN)if(mu[i])for(int j=i+i;j<MAXN;j+=i)mu[j]-
=mu[i];
}
```

6.12 sieve

```
struct Sieve {
  vector<int> primes, smallestFactor;
  Sieve(int n) {
    smallestFactor.resize(n + 1);
    for (ll i = 2; i <= n; i++) {
      if (smallestFactor[i] == 0) {
        smallestFactor[i] = i;
        primes.push_back(i);
      }
      for (ll prime : primes) {
        if (prime * i > n || prime > smallestFactor[i])
          break;
        smallestFactor[prime * i] = prime;
      }
    }
  }
  bool getIsPrime(int n) { return smallestFactor[n] == n; }
  map<int, int> factorize(int n) {
    map<int, int> factors;
    while (n > 1) {
      int factor = smallestFactor[n];
      factors[factor]++;
      n /= factor;
    }
    return factors;
  }
};
```

6.13 simplex

```
/* Solves a general linear maximization problem: maximize
$c^T x$ subject to $Ax \le b$, $x \ge 0$. Returns -inf if
there is no solution, inf if there are arbitrarily good
```

```
solutions, or the maximum value of $c^T x$ otherwise. The
input vector is set to an optimal $x$ (or in the unbounded
case, an arbitrary solution fulfilling the constraints).
Numerical stability is not guaranteed. For better
performance, define variables such that $x = 0$ is viable.
Usage:
vvd A = {{1,-1}, {-1,1}, {-1,-2}};
vd b = {1,1,-4}, c = {-1,-1}, x;
T val = LPSolver(A, b, c).solve(x);
Time: O(NM * \#pivots), where a pivot may be e.g. an edge
relaxation. O(2^n) in the general case.*/
typedef double T;//long double, Rational, double + mod<P>...
typedef vector<T> vd;
typedef vector<vd> vvd;
const T eps = 1e-8, inf = 1 / .0;
#define MP make_pair
#define ltj(X) \
    if (s == -1 || MP(X[j], N[j]) < MP(X[s], N[s])) s = j
struct LPSolver {
    int m, n; vector<int> N, B; vvd D;
    LPSolver(const vvd &A,const vd &b,const vd &c) :
m(b.size()),n(c.size()),N(n+1),B(m),D(m+2,vd(n+2)){
        rep(i, m) rep(j, n) D[i][j] = A[i][j];
        rep(i, m) {
            B[i] = n + i; D[i][n] = -1; D[i][n + 1] = b[i];
        }
        rep(j, n) { N[j] = j; D[m][j] = -c[j]; }
        N[n] = -1; D[m + 1][n] = 1;
    }
    void pivot(int r, int s) {
        T *a = D[r].data(), inv = 1 / a[s];
        rep(i, m + 2) if (i != r && abs(D[i][s]) > eps) {
            T *b = D[i].data(), inv2 = b[s] * inv;
            repx(j, 0, n + 2) b[j] -= a[j] * inv2;
            b[s] = a[s] * inv2;
        }
        rep(j, n + 2) if (j != s) D[r][j] *= inv;
        rep(i, m + 2) if (i != r) D[i][s] *= -inv;
        D[r][s] = inv;
        swap(B[r], N[s]);
    }
    bool simplex(int phase) {
        int x = m + phase - 1;
        for (;;) {
            int s = -1;
            rep(j, n + 1) if (N[j] != -phase) ltj(D[x]);
            if (D[x][s] >= -eps) return true;
            int r = -1;
            rep(i, m) {
                if (D[i][s] <= eps) continue;
                if (r == -1 || MP(D[i][n + 1] / D[i][s], B[i])
< MP(D[r][n + 1] / D[r][s], B[r])) r = i;
            }
            if (r == -1) return false;
            pivot(r, s);
        }
    }
    T solve(vd &x) {
        int r = 0;
        repx(i, 1, m) if (D[i][n + 1] < D[r][n + 1]) r = i;
        if (D[r][n + 1] < -eps) {
            pivot(r, n);
            if (!simplex(2) || D[m + 1][n + 1] < -eps) return -
inf;
            rep(i, m) if (B[i] == -1) {
                int s = 0;
                repx(j, 1, n + 1) ltj(D[i]);
                pivot(i, s);
            }
        }
        bool ok = simplex(1);
        x = vd(n);
        rep(i, m) if (B[i] < n) x[B[i]] = D[i][n + 1];
        return ok ? D[m][n + 1] : inf;
    }
};
```

6.14 system mod 2

```
struct Mod2System {
  vector<vector<int>> matrix;
  int rows, columns, variables;
  vector<int> pivotPosition, freeColumns;
  Mod2System(vector<vector<int>> matrixTosolve) {
    matrix = matrixTosolve;
    rows = matrix.size();
    columns = (rows > 0 ? matrix[0].size() : 0);
    variables = columns - 1;
    pivotPosition = vector<int>(rows, -1);
    int pivotRow = 0;
    for (int pivotColumn = 0; pivotColumn < variables &&
pivotRow < rows;
         pivotColumn++) {
      int pivotIndex = pivotRow;
      while (pivotIndex < rows && matrix[pivotIndex]
[pivotColumn] == 0)
        pivotIndex++;
      if (pivotIndex == rows)
        continue;
      swap(matrix[pivotRow], matrix[pivotIndex]);
      pivotPosition[pivotRow] = pivotColumn;
      for (int i = 0; i < rows; i++)
        if (i != pivotRow && (matrix[i][pivotColumn] & 1))
          for (int j = pivotColumn; j < columns; j++)
            matrix[i][j] = matrix[i][j] ^ matrix[pivotRow][j];
      pivotRow++;
    }
    vector<bool> isPivot(variables);
    for (int position : pivotPosition)
      if (position != -1)
        isPivot[position] = true;
    for (int j = 0; j < variables; j++)
      if (!isPivot[j])
        freeColumns.push_back(j);
  }
  vector<int> findOneSolution() {
    vector<int> solution(variables);
    if (freeColumns.empty())
      return solution;
    solution[freeColumns[0]] = 1;
    for (int i = 0; i < rows; i++) {
      if (pivotPosition[i] == -1)
        continue;
      int position = pivotPosition[i];
      solution[position] = matrix[i][freeColumns[0]];
    }
    return solution;
  }
  vector<vector<int>> findSolutions(int maxSolutions = 1e5) {
    if (freeColumns.empty())
      return {vector<int>(variables)};
    vector<vector<int>> solutions;
    for (int freeColumn : freeColumns) {
      vector<int> solution(variables);
      solution[freeColumn] = 1;
      for (int i = 0; i < rows; i++) {
        if (pivotPosition[i] == -1)
          continue;
        int pivotCol = pivotPosition[i];
        solution[pivotCol] = matrix[i][freeColumn];
      }
      solutions.push_back(solution);
      if (solutions.size() == maxSolutions)
        break;
    }
    return solutions;
  }
  bool isTrivial(vector<int> &solution) {
    return count(solution.begin(), solution.end(), 1) == 0;
  }
  bool onlyHasTrivialSolution() {
    if (freeColumns.empty())
      return true;
    vector<int> solution = findOneSolution();
    return isTrivial(solution);
  }
};
```

6.15 theorems

```
Burnside lemma
Tomemos imagenes x en X y operaciones (g: X -> X) en G.
Si #g es la cantidad de imagenes que son puntos fijos de g,
entonces la cantidad de objetos es `(sum_{g in G} #g) / |G|`
Es requisito que G tenga la operacion identidad, que toda
operacion tenga inversa y que todo par de operaciones tenga
su combinacion.

Rational root theorem
Las raices racionales de un polinomio de orden n con
coeficientes enteros A[i] son de la forma p / q, donde p y q
son coprimos, p es divisor de A[0] y q es divisor de A[n].
Notar que si A[0] = 0, cero es raiz, se puede dividir el
```

```
polinomio por x y aplica nuevamente el teorema.

Petersens theorem
Every cubic and bridgeless graph has a perfect matching.

Number of divisors for powers of 10
(0,1) (1,4) (2,12) (3,32) (4,64) (5,128) (6,240) (7,448)
(8,768) (9,1344) (10,2304) (11,4032) (12,6720) (13,10752)
(14,17280) (15,26880) (16,41472) (17,64512) (18,103680)

Kirchoff Theorem: Sea A la matriz de adyacencia del multi-
grafo (A[u][v] indica la cantidad de aristas entre u y v)
Sea D una matriz diagonal tal que D[v][v] es igual al grado
de v (considerando auto aristas y multi aristas). Sea
L = A - D. Todos los cofactores de L son iguales y equivalen
a la cantidad de Spanning Trees del grafo. Un cofactor (i,j)
de L es la multiplicación de (-1)^{i + j} con el determinant
de la matriz al quitar la fila i y la columna j

Prufer Code: Dado un árbol con los nodos indexados: busca la
hoja de menor índice, bórrala y anota el índice del nodo al
que estaba conectado. Repite el paso anterior n-2 veces. Lo
anterior muestra una biyección entre los arreglos de tamaño
n-2 con elementos en [1, n] y los árboles de n nodos, por lo
que hay n^{n-2} spanning trees en un grafo completo.
Corolario: Si tenemos k componentes de tamaños s1,s2,...,sk
entonces podemos hacerlos conexos agregando k-1 aristas
entre nodos de s1*s2*...*sk*n^{k-2} formas

Combinatoria
Catalan: C_{n+1} = sum(C_i*C_{n-i} for i \in [0, n])
Catalan: C_n = \frac{1}{n+1}*\binom{2n, n}
Sea C_n^k las formas de poner n+k pares de paréntesis, con
los primeros k paréntesis abiertos (esto es, hay 2n + 2k
carácteres), se tiene que
C_n^k = (2n+k-1)*(2n+k)/(n*(n+k+1)) * C_{n-1}^k
Sea D_n el número de permutaciones sin puntos fijos, entoces
D_n = (n-1)*(D_{n-1} + D_{n-2}), D_0 = 1, D_1 = 0
```

6.16 theorems and formulas

$$n! \sim \sqrt{2\pi n}\left(\frac{n}{e}\right)^n$$

$$\sum_{i=0}^{k}\binom{n+i}{i} = \binom{n+k+1}{k}$$

$\begin{bmatrix} n \\ k \end{bmatrix}$ = perm of n elements with k cycles

$$\begin{bmatrix} n+1 \\ k \end{bmatrix} = n\begin{bmatrix} n \\ k \end{bmatrix} + \begin{bmatrix} n \\ k-1 \end{bmatrix}$$

$\left\{ {n \atop k} \right\}$ = partitions of an n-element set into k parts

$$\left\{ {n \atop k} \right\} = \frac{1}{k!}\sum_{i=0}^{k}(-1)^i\binom{k}{i}(k-i)^n$$

$$\left\{ {n+1 \atop k} \right\} = k\left\{ {n \atop k} \right\} + \left\{ {n \atop k-1} \right\}$$

Integers $d_1 \geq \cdots \geq d_n \geq 0$ can be the degree sequence of a finite simple graph on n vertices $\Leftrightarrow d_1 + \cdots + d_n$ is even and for every k in $1 \leq k \leq n$

$$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n}\min(d_i, k)$$

$$a^n = a^{\varphi(m)+n \bmod \varphi(m)} (\bmod m) \text{ if } n > \lg(m)$$

Misere Nim: if $\exists a_i > 1$ then normal nim; else the condition is reversed.

Derangements: Num of permutations of $n = 0, 1, 2, ...$ elements without fixed points is $1, 0, 1, 2, 9, 44, 265, 1854, 14833, ...$ Recurrence: $D_n = (n-1)(D_{n-1} + D_{n-2}) = n * D_{n-1} + (-1)^n$

Collary: number of permutations with exactly k fixed points $\binom{n}{k}D_{n-k}$

Eulerian numbers: $E(n,k)$ is the number of permutations with exactly k descents ($i : \pi_i < \pi_{i+1}$), ascents ($\pi_i > \pi_{i+1}$) / excedances ($\pi > i$) / $k+1$ weak excedances ($\pi \geq i$).

$$E_{n,k} = (k+1)E_{n-1,k} + (n-k)E_{n-1,k-1}$$

Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d) g\left(\frac{n}{d}\right)$$

Other useful formulas/forms

$$\sum_{d|n}\mu(d) = [n = 1] \text{ (very useful)}$$

$$g(n) = \sum_{1\leq m\leq n} f\left(\left\lfloor \tfrac{n}{m} \right\rfloor\right) \Leftrightarrow f(n) = \sum_{1\leq m\leq n}\mu(m)g\left(\left\lfloor \tfrac{n}{m} \right\rfloor\right)$$

$$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d}\mu\left(\tfrac{d}{n}\right)g(d)$$

Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1, \quad p(n) = \sum_{k\in\mathbb{Z}\setminus\{0\}} (-1)^{k+1} p\left(n - \frac{k(3k-1)}{2}\right)$$

$$p(n) \approx \frac{0.145}{n} \cdot e^{2.56\sqrt{n}}$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	$\approx 2e5$	$\approx 2e8$

Bell numbers

Total number of partitions of n distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, ...$

For p prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod p$$

Labeled unrooted trees

on n vertices: n^{n-2}

on k existing trees of size n_i: $n_1 n_2 \cdots n_k n^{k-2}$

with degrees d_i: $\frac{(n-2)!}{(d_1-1)!\cdots(d_n-1)!}$

Catalan numbers

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1, \quad C_{n+1} = \frac{2(2n+1)}{n+2}C_n, \quad C_{n+1} = \sum C_i C_{n-i}$$

$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, ...$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

Narayana Numbers

$$N(n,k) = \frac{1}{n}\binom{n}{k}\binom{n}{k-1}$$

6.17 prefix xor

```cpp
// Calculates XOR from 1 to n
// Complexity: O(1)
// NOTE: use long long of int128 if needed
int prefixXOR(int n) {
  // If n is a multiple of 4
  if (n % 4 == 0)
    return n;
  // If n%4 gives remainder 1
  if (n % 4 == 1)
    return 1;
  // If n%4 gives remainder 2
  if (n % 4 == 2)
    return n + 1;
  // If n%4 gives remainder 3
  return 0;
}
```

7 Strings

7.1 aho corasick

```cpp
struct AC
{
    ll c = 0, ec = 0, M, A, af = -1;
    vector<vl> N, G; vl L, E;
    vector<vl> val;
    vl vis, cnt;
    // L -> suffix link G -> anti L
    // E -> string finish
    AC (ll M, ll A) : M(M), A(A), N(M, vl(A, 0)), G(M, vl(0)),
E(M, 0), L(M, 0), val(M, vl(0)), vis(M, 0), cnt(M, 0){}
    ll add(string s, int id){ // return endpoint
        af++;
        ll p = 0;
        for (char l : s){
            int t = l - 'a';
            if (!N[p][t]) N[p][t] = ++c;
            p = N[p][t];
        }
        val[p].push_back(id);
        return p;
    }
    // construct aho corasick structure
    void init(){
        queue<int> q; q.push(0); L[0] = -1;
        while (!q.empty()){
            int p = q.front(); q.pop();
            for(int c = 0; c < A; c++){
                int u = N[p][c]; if (!u) continue;
                L[u] = L[p] == -1 ? 0 : N[L[p]][c], q.push(u);
                G[L[u]].push_back(u);
            }
            if (p) for(int c = 0; c < A; c++) if (!N[p][c])
N[p][c] = N[L[p]][c];
        }
    }
    // para un texto s, recorrer las aristas por el arbol
    void run(string &s){
        for(int p = 0, i = 0; i<(int)s.size(); ++i){
            int t = s[i]-'a';
            p = (N[p][t] == -1 ? 0 : N[p][t]);
            cnt[p]++;
        }
    }
    void dfs(int u = 0){
        vis[u] = true;
        for(int v : G[u]){
            if(v == -1) continue;
            if(!vis[v]) dfs(v);
            cnt[u] += cnt[v];
        }
        for(int v : val[u]){
            ans[v] = cnt[u];
        }
    }
};
```

7.2 hashing

```cpp
struct RH {
    int binpow(int a, int b, int MOD) {
        int res = 1;
        while(b) {
            if(b & 1) res = (1LL * res * a) % MOD;
            a = (1LL * a * a) % MOD;
            b >>= 1;
        }
        return res;
    }
    // choose base B random to avoid hacks 33 37 41
    // randomize V(s[i])
    const int B = 119, M[2] = {999727999, 1070777777}, P[2] =
{325255434, 10018302};
    vector<int> H[2], I[2];
    RH(string &s) {
        int N = s.size();
        rep(k, 2) {
            H[k].resize(N + 1), I[k].resize(N + 1);
            H[k][0] = 0; ll b = 1;
            rep(i, N) {
                H[k][i + 1] = (H[k][i] + b * (s[i] - 'a' + 1))
% M[k];
                b = (b * B) % M[k];
            }
            I[k][N] = binpow(b, M[k] - 2, M[k]);
            for(int i = N - 1; i >= 0; i--) I[k][i] = (1LL *
I[k][i + 1] * B) % M[k];
        }
    }
    ll get(int l, int r)  // inclusive - exclusive
    {
        ll h0 = (H[0][r] - H[0][l] + M[0]) % M[0];
        h0 = (1LL * h0 * I[0][l]) % M[0];
        ll h1 = (H[1][r] - H[1][l] + M[1]) % M[1];
        h1 = (1LL * h1 * I[1][l]) % M[1];
        return (h0 << 32) | h1;
    }
};
```

7.3 hashing2d

```cpp
struct Hashing2D {
    vector<vector<int>> hs;
    vector<int> PWX, PWY;
    int n, m;
    static const int PX = 3731,  PY = 2999, mod = 998244353;
    Hashing() {}
    Hashing(vector<string>& s) {
        n = (int)s.size(), m = (int)s[0].size();
        hs.assign(n + 1, vector<int>(m + 1, 0));
        PWX.assign(n + 1, 1);
        PWY.assign(m + 1, 1);
        for (int i = 0; i < n; i++) PWX[i + 1] = 1LL * PWX[i] *
PX % mod;
        for (int i = 0; i < m; i++) PWY[i + 1] = 1LL * PWY[i] *
PY % mod;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                hs[i + 1][j + 1] = s[i][j] - 'a' + 1;
            }
        }
        for (int i = 0; i <= n; i++) {
            for (int j = 0; j < m; j++) {
                hs[i][j + 1] = (hs[i][j + 1] + 1LL * hs[i][j] *
PY % mod) % mod;
            }
        }
        for (int i = 0; i < n; i++) {
            for (int j = 0; j <= m; j++) {
                hs[i + 1][j] = (hs[i + 1][j] + 1LL * hs[i][j] *
PX % mod) % mod;
            }
        }
    }
    int get_hash(int x1, int y1, int x2, int y2) { // 1-indexed
        assert(1 <= x1 && x1 <= x2 && x2 <= n);
        assert(1 <= y1 && y1 <= y2 && y2 <= m);
        x1--;
        y1--;
        int dx = x2 - x1, dy = y2 - y1;
        return (1LL * (hs[x2][y2] - 1LL * hs[x2][y1] * PWY[dy]
% mod + mod) % mod -
            1LL * (hs[x1][y2] - 1LL * hs[x1][y1] * PWY[dy] %
mod + mod) % mod * PWX[dx] % mod + mod) % mod;
    }
    int get_hash() {
        return get_hash(1, 1, n, m);
    }
```

```
    }
};
```

7.4 kmp

```
vector<int> kmppre(string& t){ // r[i]: longest border of
t[0,i)
  vector<int> r(t.size()+1);r[0]=-1;
  int j=-1;
  fore(i,0,t.size()){
    while(j>=0&&t[i]!=t[j])j=r[j];
    r[i+1]=++j;
  }
  return r;
}
void kmp(string& s, string& t){ // find t in s
  int j=0;vector<int> b=kmppre(t);
  fore(i,0,s.size()){
    while(j>=0&&s[i]!=t[j])j=b[j];
    if(++j==t.size())printf("Match at %d\n",i-j+1),j=b[j];
  }
}
```

7.5 manacher

```
int n;
string s;
int main(){
    vi d1(n); // odd sized palindromes
    for (int i = 0, l = 0, r = -1; i < n; i++) {
        int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
        while (0 <= i - k && i + k < n && s[i - k] == s[i + k])
k++;
        d1[i] = k--;
        if (i + k > r) l = i - k, r = i + k;
    }
    vi d2(n); // even sized palindromes (center to the right)
    for (int i = 0, l = 0, r = -1; i < n; i++) {
        int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i +
1);
        while (0 <= i - k - 1 && i + k < n && s[i - k - 1] ==
s[i + k]) k++;
        d2[i] = k--;
        if (i + k > r) l = i - k - 1, r = i + k;
    }
}
```

7.6 palindromic tree

```
struct Node {    // (*) = Optional
    int len;     // length of substring
    int to[26];  // insertion edge for all characters a-z
    int link;    // maximun palindromic suffix
    int i;       // (*) start index of current Node
    int cnt;     // (*) # of occurrences of this substring
    Node(int len, int link=0, int i=0, int cnt=1): len(len),
    link(link), i(i), cnt(cnt) {memset(to, 0, sizeof(to));}
}; struct EerTree {    // Palindromic Tree
    vector<Node> t; // tree (max size of tree is n+2)
    int last;       // current node
    EerTree(string &s) : last(0) {
        t.emplace_back(-1); t.emplace_back(0); // root 1 & 2
        rep(i, s.size()) add(i, s);  // construct tree
        for(int i = t.size()-1; i > 1; i--)
            t[t[i].link].cnt += t[i].cnt;
    }
    void add(int i, string &s){         // vangrind warning:
        int p=last, c=s[i]-'a';         // i-t[p].len-1 = -1
        while(s[i-t[p].len-1] != s[i]) p = t[p].link;
        if(t[p].to[c]){ last = t[p].to[c]; t[last].cnt++; }
        else{
            int q = t[p].link;
            while(s[i-t[q].len-1] != s[i]) q = t[q].link;
            q = max(1, t[q].to[c]);
            last = t[p].to[c] = t.size();
            t.emplace_back(t[p].len + 2, q, i-t[p].len-1);
        }
    }
};
void main(){
    string s = "abcbab"; EerTree pt(s); // build EerTree
    repx(i, 2, pt.t.size()){// list all distinct palindromes
        repx(j,pt.t[i].i,pt.t[i].i+pt.t[i].len)cout << s[j];
        cout << " " << pt.t[i].cnt << endl;
    }
}
```

7.7 rolling hash

```
struct RollingHash {
  static const ll BASE = 31;
  static const ll MOD1 = 1e9 + 9, MOD2 = 1e9 + 7, MOD3 = (119LL
<< 23) | 1;
  const vector<ll> MODS = {MOD1, MOD2, MOD3};
  vector<vector<ll>> prefixes, powers;
  int n;
  RollingHash(const string &text) {
    n = text.size();
    prefixes.assign(3, vector<ll>(n + 2));
    powers.assign(3, vector<ll>(n + 2, 1));
    for (int j = 0; j < 3; j++)
      for (int i = 1; i <= n + 1; i++)
        powers[j][i] = (powers[j][i - 1] * BASE) % MODS[j];
    for (int j = 0; j < 3; j++) {
      for (int i = 0; i < n; i++)
        prefixes[j][i + 1] = (prefixes[j][i] * BASE + text[i])
% MODS[j];
      prefixes[j][n + 1] = (prefixes[j][n] * BASE) % MODS[j];
    }
  }
  tuple<int, int, int> query(int left, int right) {
    vector<int> result(3);
    for (int i = 0; i < 3; i++) {
      ll hash = (prefixes[i][right + 1] -
                 prefixes[i][left] * powers[i][right - left +
1] + MODS[i]) %
                MODS[i];
      if (hash < 0)
        hash += MODS[i];
      result[i] = hash;
    }
    return {result[0], result[1], result[2]};
  }
};
```

7.8 suffix array

```
#define rep(i, n) for(int i = 0; i<(int)n; ++i)
#define repx(i, a, b) for(int i = (int)a; i<(int)b; ++i)
#define invrep(i, b, a) for(int i = (int)b; i>=a; --i)
#define vl vector<ll>
struct SA {
    int n; vl counts, rank, rank_, sa, sa_, lcp; // lcp is
optional
    inline int gr(int i) { return i < n ? rank[i]: 0; }
    void csort(int maxv, int k) {
        counts.assign(maxv+1, 0);
        repx(i,0,n) counts[gr(i+k)]++;
        repx(i,1,maxv+1) counts[i] += counts[i-1];
        invrep(i,n-1,0) sa_[--counts[gr(sa[i]+k)]] = sa[i];
        sa.swap(sa_);
    }
    void get_sa(vl& s) {
        repx(i,0,n) sa[i] = i;
        sort(sa.begin(), sa.end(), [&s](int i, int j) { return
s[i] < s[j]; });
        int r = rank[sa[0]] = 1;
        repx(i,1,n) rank[sa[i]] = (s[sa[i]] != s[sa[i-1]]) ? +
+r : r;
        for (int h=1; h < n and r < n; h <<= 1) {
            csort(r, h); csort(r, 0); r = rank_[sa[0]] = 1;
            repx(i,1,n) {
                if (rank[sa[i]] != rank[sa[i-1]] or
                    gr(sa[i]+h) != gr(sa[i-1]+h)) ++r;
                rank_[sa[i]] = r;
            } rank.swap(rank_);
        }
    }
    // LCP construction in O(N) using Kasai's algorithm
    // reference: https://codeforces.com/blog/entry/12796?#
comment-175287
    void get_lcp(vl& s) { // lcp is optional
        lcp.assign(n, 0); int k = 0;
        repx(i,0,n) {
            int r = rank[i]-1;
            if (r == n-1) { k = 0; continue; }
            int j = sa[r+1];
            while (i+k<n and j+k<n and s[i+k] == s[j+k]) k++;
            lcp[r] = k;
            if (k) k--;
        }
    }
    SA(vl& s) {
        n = s.size();
        rank.resize(n); rank_.resize(n);
        sa.resize(n); sa_.resize(n);
```

```
        get_sa(s); get_lcp(s); // lcp is optional
    }
};
int main() { // how to use
    string test; cin >> test;
    vl s;
    for (char c : test) s.push_back(c);
    SA sa(s);
    for (int i : sa.sa) cout << i << ":\t" << test.substr(i) <<
'\n';
    repx(i,0,s.size()) {
        printf("LCP between %d and %d is %d\n", i, i+1,
sa.lcp[i]);
    }
}
```

7.9 suffix automaton

```
struct State {int len, link; map<char,int> next; };
State st[2*MAXN]; int sz, last;              // clear next!!
void sa_init(){ last=st[0].len=0; sz=1; st[0].link=-1; }
void sa_extend(char c){// total build O(n log alphabet_size)
    int k = sz++, p; st[k].len = st[last].len + 1;
    for(p=last; p!=-1 && !st[p].next.count(c); p=st[p].link)
        st[p].next[c] = k;
    if(p == -1) st[k].link = 0;
    else {
        int q = st[p].next[c];
        if(st[p].len + 1 == st[q].len) st[k].link = q;
        else {
            int w = sz++; st[w].len = st[p].len + 1;
            st[w].next=st[q].next; st[w].link=st[q].link;
            for(; p!=-1 && st[p].next[c]==q; p=st[p].link)
                st[p].next[c] = w;
            st[q].link=st[k].link = w;
        }
    }
    last = k;
} // # states <= 2n-1 && transitions <= 3n-4 (for n > 2)
// Follow link from `last` to 0, nodes on path are terminal
// # matches = # paths from state to a terminal node
// # substrings = # paths from 0 to any node
// # substrings = sum of (len - len(link)) for all nodes
```

7.10 trie

```
struct Trie{
    static const int MAX = 1e6;
    int N[MAX][26] = {0}, S[MAX] = {0}, c = 0;
    void add(string s, int a = 1){
        int p = 0; S[p] += a;
        for (char l : s){
            int t = l - 'a';
            if (!N[p][t]) N[p][t] = ++c;
            S[p = N[p][t]] += a;
        }
    }
};
struct TrieXOR{
    static const int MAX = 1e6;
    int N[MAX][2] = {0}, S[MAX] = {0}, c = 0;
    void add(int x, int a = 1){
        int p = 0; S[p] += a;
        rep(i, 31){
            int t = (x >> (30 - i)) & 1;
            if (!N[p][t]) N[p][t] = ++c;
            S[p = N[p][t]] += a;
        }
    }
    int get(int x){
        if (!S[0]) return -1;
        int p = 0; rep(i, 31){
            int t = ((x >> (30 - i)) & 1) ^ 1;
            if (!N[p][t] || !S[N[p][t]]) t ^= 1;
            p = N[p][t]; if (t) x ^= (1 << (30 - i));
        }
        return x;
    }
};
struct Trie {
    vector<vector<int>> g;
    vector<int> count;
    int vocab;
    Trie(int vocab, int maxdepth = 10000) : vocab(vocab) {
        g.reserve(maxdepth);
        g.emplace_back(vocab, -1);
        count.reserve(maxdepth);
        count.push_back(0);
    }
    int move_to(int u, int c) {
        assert (0 <= c and c < vocab);
        int& v = g[u][c];
        if (v == -1) {
            v = g.size();
            g.emplace_back(vocab, -1);
            count.push_back(0);
        }
        count[v]++;
        return v;
    }
    void insert(const string& s, char ref = 'a') {  // insert
string
        int u = 0; for (char c : s) u = move_to(u, c - ref);
    }
    void insert(vector<int>& s) { // insert vector<int>
        int u = 0; for (int c : s) u = move_to(u, c);
    }
    db query(const string& s, char ref = 'a'){
      int u = 0;
      db cost = 0;
      for (char c : s){
        ll co = 0;
        for(auto it : g[u]) if(it != -1)co++;
        ll nex = move_to(u, c - ref);
        if(u == 0 || co > 1 || count[u] != count[nex]) cost++;
        u = nex;
      }
      return cost;
    }
    ll dfs(int u, int depht){
        ll ans = INF;
        if(count1[u] == 1 && count2[u] == 1)ans = depht;
        for(int i = 0; i < 26; i++){
            if(g[u][i] != -1) ans = min(ans, dfs(g[u][i], depht
+ 1));
        }
        return ans;
    }
    int size() { return g.size(); }
};
```

8 General

8.1 template

```
// Maybe remove unroll-loops, might increase code size too much
and lead to
// instruction cache misses.
#pragma GCC optimize("O3,unroll-loops")
// avx2 can be changed to avx, sse, sse2, sse3, sse4, see4.1,
see4.2
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
#include <bits/stdc++.h>
using namespace std;
#define SZ(x) (int)x.size()
#define rep(i, n) for (int i = 0; i < n; i++)
#define repx(i, a, b) for (int i = a; i < (int)b; ++i)
#define vl vector<ll>
#define vi vector<int>
using ll = long long;
using pii = pair<int, int>;
// mt19937_64
rng(chrono::steady_clock::now().time_since_epoch().count());
void solve() { ; }
int main() {
  ios::sync_with_stdio(0);
  cin.tie(0);
  int t = 1;
  // cin >> t;
  while (t--)
    solve();
}
```

8.2 run

```
#!/bin/bash

g++ -g -O2 -std=c++20 -pedantic -Wall -Wextra -o tmp $1.cpp
echo running
valgrind -q ./tmp
```